From inference to integration: Formal specifications as the verification spine of AI-native software development

Brendan Edmonds

BSc, MSc

A thesis submitted for the degree of Master of Philosophy at The University of Queensland in 2026

School of Electrical Engineering and Computer Science
UQ Cyber

ORCID: 0009-0007-9067-3186

Abstract

Formal specifications make software behaviour machine-checkable, yet they are rarely written in practice because authoring them by hand is costly and demands expertise most development teams lack. The machinery for *checking* programs against contracts is mature; what has been missing is an economical supply of the contracts. This thesis argues that the obstacle is not the value of specifications but their cost of acquisition, and that specifications inferred automatically from code—once cheap enough to be useful—can be composed to reason about library compatibility and integrated as the verification layer of the AI-native development lifecycles now emerging around large language models.

The work is built on a single instrument: the JML-Inferrer, a static analysis tool that derives Java Modeling Language contracts—preconditions, postconditions, loop and class invariants, and frame conditions—from Java source by heuristics over the abstract syntax tree, and holds them to the demanding standard of discharge by the OpenJML extended static checker and the Z3 SMT solver. The thesis develops three connected results. First, it establishes that heuristic, partial, automatically inferred specifications are of high enough fidelity to be useful, and that supplying them to a large language model measurably improves the unit tests the model generates, converting the abstract value of a contract into a measured downstream benefit. Second, it shows that a compositional weakest-precondition pass over the call graph strictly extends single-pass inference, recovering a substantial number of additional well-formed preconditions and enabling behavioural version-compatibility to be treated as a static, run-free analysis: a change to a callee's contract can be propagated to its callers to compute the blast radius of the change, turning semantic versioning from an asserted convention into a checkable property. Third, it shows how inferred, machine-checkable contracts supply what current agentic, specification-driven lifecycles lack—a contract layer that is machine-checkable rather than natural language, and a closed verification loop rather than an open generate-and-review cycle—with the same contract serving as both the input an agent builds against and the oracle its output is checked against.

Throughout, the thesis is scrupulous to distinguish *verified* specifications, which the checker has discharged, from those that are merely *well-formed*. Taken together, the results trace a single arc from inference—making specifications cheap—to integration—making them load-bearing in how software is built.

Declaration by author

This thesis is composed of my original work, and contains no material previously published or written by another person except where due reference has been made in the text. I have clearly stated the contribution by others to jointly-authored works that I have included in my thesis.

I have clearly stated the contribution of others to my thesis as a whole, including statistical assistance, survey design, data analysis, significant technical procedures, professional editorial advice, financial support and any other original research work used or reported in my thesis. The content of my thesis is the result of work I have carried out since the commencement of my Higher Degree by Research candidature and does not include a substantial part of work that has been submitted to qualify for the award of any other degree or diploma in any university or other tertiary institution. I have clearly stated which parts of my thesis, if any, have been submitted to qualify for another award.

I acknowledge that an electronic copy of my thesis must be lodged with the University Library and, subject to the policy and procedures of The University of Queensland, the thesis be made available for research and study in accordance with the Copyright Act 1968 unless a period of embargo has been approved by the Dean of the Graduate School.

I acknowledge that copyright of all material contained in my thesis resides with the copyright holder(s) of that material. Where appropriate I have obtained copyright permission from the copyright holder to reproduce material in this thesis and have sought permission from co-authors for any jointly authored works included in the thesis.

Brendan Edmonds 2026

Statement of contributions to jointly authored works

This thesis is a traditional monograph and contains no jointly authored works reproduced as chapters. All work reported in it was conceived, designed, implemented, analysed and written by the candidate, Brendan Edmonds, under the supervision of Professor Mark Utting (principal advisor) and Dr Guowei Yang (associate advisor), who contributed conceptual guidance and critical review. Where the thesis draws on manuscripts prepared for publication during the candidature, those manuscripts were led by the candidate and their material has been rewritten and condensed for the thesis; the chapters are the candidate's original work.

Publications during candidature

This thesis is presented as a traditional monograph. No publications are reproduced as chapters; the chapters are the candidate's original, integrated work, drawing where relevant on manuscripts prepared for publication during the candidature and rewritten to form a single coherent argument.

Publications included in this thesis

None. The thesis does not incorporate any publication as a chapter.

Submitted manuscripts included in this thesis

None.

Other publications during candidature

The following manuscripts were prepared during the candidature. Their material, where used, has been rewritten and condensed for the thesis.

- B. Edmonds and M. Utting. *Do JML Specifications Help LLMs Write Better Unit Tests?* In preparation for the *Journal of Software: Evolution and Process*.
- B. Edmonds and M. Utting. *Does Compositional Refinement Strictly Extend Heuristic Specification Inference? A Measurement Study.* Under review, *International Conference on Software Engineering (ICSE)*.
- B. Edmonds and M. Utting. *Embedding JML Specifications in Compiled Java Artefacts and OpenAPI Documents.* In preparation for the *Journal of Software: Evolution and Process*.
- B. Edmonds and M. Utting. *Specification Inference in Continuous Integration: Design and a Reference Implementation.* In preparation for the *Journal of Software: Evolution and Process*.
- B. Edmonds and M. Utting. *A Verification-Centric Reference Pipeline for AI-Native Software Development.* In preparation for the *Journal of Software: Evolution and Process*.

Contributions by others to the thesis

Professor Mark Utting and Dr Guowei Yang contributed to the conception and design of the research programme and provided critical review. No other persons contributed to the work reported in this thesis.

Statement of parts of the thesis submitted to qualify for the award of another degree

No works submitted towards another degree have been included in this thesis.

Research involving human or animal subjects

No animal or human subjects were involved in the research reported in this thesis. The industrial case study outlined as future work would involve human participants and is contingent on prior approval from the relevant Human Research Ethics Committee.

Acknowledgements

I am deeply grateful to my principal advisor, Professor Mark Utting, whose depth in formal methods and unfailing rigour shaped every part of this work, and to my associate advisor, Dr Guowei Yang, for his guidance and perspective throughout the candidature.

I thank the School of Electrical Engineering and Computer Science and the UQ Cyber initiative for providing the environment and resources that made this research possible, and the maintainers of OpenJML, the Z3 SMT solver, JavaParser, and Apache Commons Lang, whose open tools and corpora underpin the experiments reported here.

Finally, I thank my family for their patience and support over the course of this work.

Financial support

This research was supported by an Australian Government Research Training Program (RTP) Scholarship and by the University of Queensland Cyber initiative.

Keywords

specification inference, formal verification, Java Modeling Language, large language models, design by contract, software development lifecycle, behavioural compatibility, brownfield software

Australian and New Zealand Standard Research Classifications (ANZSRC)

ANZSRC code: 461304, Software engineering, 70%
ANZSRC code: 461399, Theory of computation not elsewhere classified, 20%
ANZSRC code: 461103, Deep learning, 10%

Fields of Research (FoR) Classification

FoR code: 4613, Theory of computation, 40%
FoR code: 4612, Software engineering, 60%

Contents

List of Figures

List of Tables

List of Abbreviations

AST	Abstract Syntax Tree
AI-DLC	AI-Driven Development Lifecycle
CEGIS	Counterexample-Guided Inductive Synthesis
CI	Continuous Integration
DbC	Design by Contract
ESC	Extended Static Checking
JML	Java Modeling Language
LLM	Large Language Model
SMT	Satisfiability Modulo Theories
SP	Strongest Postcondition
WP	Weakest Precondition

Chapter 1

Introduction

1.1 When code becomes cheap, the specification becomes scarce

For most of the history of software engineering, source code has been the scarce, durable, and authoritative artefact of a project. Code was expensive to produce and to change; it accreted the team's institutional memory—the handling of an awkward edge case, the workaround for a third-party defect, the carefully tuned boundary of a loop—and it tended to outlive the informal notes that described it. The intent behind the code, what the program was *supposed* to do, was usually recorded only informally and routinely allowed to drift out of step with the implementation.

The maturation of large language models for code generation disturbs this settled economics. Systems trained on vast corpora can now produce working implementations from natural-language descriptions [13, 40], and agentic variants can navigate a repository, edit files, run a build, and iterate toward a passing test suite with limited human intervention [34, 62]. As the marginal cost of producing a plausible implementation falls toward zero, the relative value of a project's artefacts inverts. If code can be regenerated cheaply from a sufficiently precise description of intent, it becomes a disposable by-product, and the durable, scarce artefact—the thing that must be preserved, reviewed, and versioned—becomes the precise *specification* against which code is generated and judged. The claim is deliberately conditional: *to the extent that* code generation becomes routine, the bottleneck migrates from writing code to specifying intent precisely enough that generated code can be trusted. Model output is fluent but unreliable; it hallucinates interfaces and produces code that passes a weak test suite while remaining subtly wrong [41, 64]. The discipline's traditional answer to "how do we know this code is correct?"—read it carefully—scales poorly when code is produced faster than it can be read. What is required instead is a machine-checkable statement of what the code must do, and a mechanical means of checking it. That statement is a formal specification.

1.2 The barrier: specifications are valuable but unwritten

That programs should carry precise, machine-checkable statements of their intended behaviour is among the oldest aspirations of the field. Hoare's axiomatic basis [30] and Dijkstra's discipline of programming [21] established preconditions, postconditions, and loop invariants as the vocabulary

of correctness; Meyer's design by contract [45] made the contract, rather than the implementation, the unit of interface between components—precisely the property that lets an implementation be regenerated without disturbing its callers, provided the contract is preserved [48]. The Java Modeling Language (JML) gave this discipline a tool-supported notation [38], and modern verifiers such as OpenJML [15], discharging obligations through SMT solvers such as Z3 [18], can check non-trivial Java methods against JML contracts by extended static checking without running the program. The machinery for *checking* programs against contracts is mature; what has been missing is an economical supply of the contracts.

Despite this lineage, formal specifications remain rare in industrial codebases. This is an *adoption* problem, not a *value* problem. Few engineers dispute that a precise contract is useful; many never write one, because authoring specifications is costly, demands unevenly distributed expertise, and yields benefits that are diffuse and deferred while the costs are immediate and concentrated on the author. Faced with this asymmetry, the rational engineer under schedule pressure writes a unit test, or nothing. The same asymmetry recurs for whole codebases: legacy systems—most systems that matter—are precisely those whose behaviour is no longer fully understood [24], and the cost of retrofitting specifications by hand is prohibitive in exactly the cases where they would be most valuable. This diagnosis points at the leverage point. If the dominant obstacle is the cost of *authoring* specifications, then lowering that cost—ideally to near zero, by inferring specifications automatically from existing code—should change the calculus of adoption. Automatic inference is itself a mature line of research: Daikon infers likely invariants from executions [23], and Houdini guesses candidate annotations and prunes those that fail to verify [25]. What they leave open is whether inference can be made cheap and unobtrusive enough to run as a matter of course, whether its output is genuinely useful downstream, and whether the resulting specifications can be composed and integrated into how software is built.

1.3 Why now: agentic lifecycles without a verification loop

The case for cheap specifications would be academic were it not for a concurrent shift in how software is built. A new family of development lifecycles has emerged—AWS's AI-Driven Development Lifecycle [3], the GitHub Spec Kit [29], and AWS's Kiro [4]—in which a developer writes a specification of desired behaviour, an agent plans and implements against it, and the cycle repeats. These lifecycles at last place the specification at the centre of the workflow, but they contain a structural gap: the specifications that drive them are written in *natural language*, which is not machine-checkable. The verification step, where it exists, falls back on tests synthesised by the same fallible model against the same ambiguous description, and on human review—the very bottleneck the lifecycle was meant to relieve. The agentic *capability* was demonstrated first on benchmarks where a hidden test suite supplies the oracle for free [34]; the industrial lifecycles transplant the capability but leave the oracle behind. The result is a loop that generates code quickly but cannot *verify* it: there is no machine-checkable contract layer and no closed loop in which a candidate is checked against a precise oracle and rejected when it fails.

The compatibility dimension of this gap deserves emphasis. Modern software is assembled from libraries, and the contract between a library and its clients is communicated by a version number under the convention of semantic versioning [53]. This convention is honoured unevenly—breaking changes are routinely shipped in releases the version number declares compatible [54]—because the contract being versioned is implicit: there is no machine-checkable statement of what a library guarantees, and so no mechanical way to decide whether a new release still honours what the old one promised. An agentic lifecycle that regenerates components against natural-language descriptions inherits and amplifies this problem, as each regeneration is a potential silent breaking change. A compositional, machine-checkable specification is exactly what turns version compatibility from an asserted convention into a checkable property.

1.4 Thesis statement and contributions

The central claim of this thesis is that **formal specifications are valuable but unwritten because authoring them is costly; automatic inference makes them cheap enough first to be useful, then to be composed to reason about library compatibility, and finally to serve as the contract layer and verification loop of AI-native development lifecycles.** The two animating concerns, led with throughout, are formal-methods *adoption*—the removal of the authoring barrier through inference—and library and version *compatibility*—the use of compositional, automatically inferred specifications to reason about whether a change to a component preserves the behaviour its callers depend upon. Two words guard the statement. "Cheap" does not mean free of judgement: inference removes the up-front authoring cost by beginning from a verified draft rather than a blank page. And where the thesis proposes a lifecycle architecture, it is careful to distinguish what has been *measured* from what has been *designed*.

The programme is organised around three research questions that trace this arc. **RQ1 (fidelity and utility):** can JML contracts be inferred automatically with enough fidelity to be useful, and do they measurably improve a downstream task such as large-language-model test generation? **RQ2 (compositional compatibility):** does a compositional weakest-precondition pass extend single-pass inference, and does it let behavioural version compatibility be decided as a static analysis across a call graph? **RQ3 (lifecycle integration):** can inferred, machine-checkable contracts serve as the contract layer and verification loop of AI-native development lifecycles? The three contributions that follow answer RQ1, RQ2, and RQ3 respectively.

The thesis makes three contributions, each developed and evaluated in its own chapter:

1. **Inferred specifications are useful, and improve LLM test generation** (chapter 3). The JML-Inferrer recovers sound, verifiable contracts from Java source by AST heuristics; a controlled comparison shows that supplying these contracts to a large language model measurably improves the unit tests it generates, relative to the same model working from code alone. This establishes that heuristic, partial, automatically inferred specifications are nonetheless good enough to deliver downstream utility.

2. **Compositional refinement strictly extends inference and enables version-compatibility reasoning** (chapter 4). A second, weakest-precondition pass over the call graph recovers well-formed preconditions that single-pass heuristic inference cannot, and lets a change to a callee's contract be propagated to its callers—turning behavioural version compatibility into a static, run-free analysis.

3. **Inferred contracts as the verification spine of AI-native development** (chapter 5). A reference architecture in which a machine-checkable contract is the pivot between ambiguous human intent and precise machine verification, serving as both the input an agent builds against and the oracle its output is checked against, and supplying the closed verification loop that current agentic lifecycles lack.

1.5 The instrument: the JML-Inferrer

A single instrument gives the programme its empirical coherence. The JML-Inferrer is a static analysis tool, implemented in Java 21 over the JavaParser abstract syntax tree library, that infers JML contracts directly from Java source by heuristics that pattern-match against the syntax tree. Inferred contracts are checked by OpenJML's extended static checking [15], with obligations discharged by Z3 [18]. A static, syntax-driven approach requires no test inputs, instrumentation, or execution traces, so it applies to a codebase exactly as it sits in a repository; and because the heuristics are deterministic functions of the syntax, the same input yields the same contracts. The heuristics are unapologetically partial: no claim is made that a recovered contract is complete, only that it is sound, useful, and verifiable. The discipline that keeps them honest is verification—a candidate contract that OpenJML cannot discharge is treated as a defect in the inference to be fixed, not as an acceptable approximation to be shipped. Throughout, the thesis distinguishes a *verified* specification, which the checker has discharged, from one that is merely *well-formed*, which is syntactically valid and scope-correct but has not been through that check, and it never allows the distinction to blur.

1.6 Thesis structure

Chapter 2 establishes the conceptual and technical foundations—design by contract and its axiomatic antecedents, JML and OpenJML, the SMT machinery, the prior art in specification inference, and the emerging landscape of LLM code generation and agentic development—and situates the work against related research. Chapter 3 presents the JML-Inferrer in detail and reports the fidelity and downstream-utility results (the first contribution). Chapter 4 develops the compositional weakest-precondition pass and its application to version-compatibility reasoning (the second contribution). Chapter 5 assembles the validated capabilities into a verification-centric reference pipeline for AI-native development (the third contribution). Chapter 6 discusses the findings and the threats to their validity, and chapter 7 summarises the contributions, states the limitations—including the boundary between verified and merely well-formed specifications—and sets out directions for future work.

Chapter 2

Background and Related Work

This chapter establishes the foundations on which the thesis builds and situates the work against the prior art. It draws on four largely separate lines of research—deductive program verification, specification inference, large-language-model (LLM) software engineering, and the empirical study of software evolution—and argues that none, alone, spans the problem addressed here. The survey is critical rather than encyclopaedic, naming what each tradition achieves and where it breaks down, so that the gap emerges by subtraction and section 2.6 can state the thesis's position within it.

2.1 Design by contract and its axiomatic antecedents

A *specification* is a statement of intended behaviour independent of any implementation. The formal-methods tradition renders it as a predicate, or a pair of predicates, over program states, and every contract here descends from it. The axiomatic foundation is Hoare logic, in which the triple $\{P\}\,S\,\{Q\}$ asserts that if precondition P holds before command S executes and S terminates, then postcondition Q holds afterward [30]. As Hoare defined it the triple expresses *partial* correctness—it promises nothing about termination, only that *if* S halts, Q holds—a distinction that recurs throughout, since the extended static checking used here is a partial-correctness regime. Dijkstra recast this as a calculation: the weakest precondition $\mathrm{wp}(S, Q)$ is the weakest predicate on the initial state guaranteeing that S terminates and establishes Q [20, 21]. The calculus matters structurally as well as historically: because wp is a computable function of the postcondition and the statement, a change to the postcondition a callee guarantees can be pushed back mechanically through its call sites—the technical heart of the compositional change-propagation analysis of chapter 4.

The refinement calculus extended this into a development method that transforms an abstract specification into code by provably correct steps [7, 47]—the ancestor of program synthesis and, through it, of the contract-to-code generation LLM agents now perform. The decisive difference is that refinement *trusts the derivation*, each step justified by a proof rule, whereas an LLM agent offers no such justification. The thesis therefore inverts the relationship, letting a fallible generator propose an implementation freely and recovering soundness by *verifying* the result against the contract rather than trusting the process that produced it.

Design by contract, introduced by Meyer for Eiffel, operationalised these semantics for object-

oriented software [45, 46]. A method is governed by a precondition the caller must establish, a postcondition the method guarantees in return, and class invariants every public method must preserve. The conceptual contribution is the framing of behaviour as mutual *obligations* between caller and callee: the caller must satisfy the precondition and benefits from the postcondition; the callee may assume the precondition and must deliver the postcondition. This duality is foundational, because it is exactly what makes contracts *compose*—a change to a callee's contract recomputes every caller's obligations, the property the compatibility analysis exploits. It carries a further consequence: if the contract is the authoritative statement of intent and the code merely its fulfilment, then when code can be regenerated cheaply the contract, not the code, is the scarce and durable asset. Eiffel's limitation is its medium—assertions are evaluated at runtime against whatever inputs occur, so the guarantee is sampled rather than universal.

The behavioural interface specification tradition closes that gap. The Java Modeling Language (JML) is the contract notation adopted throughout [38]. Its annotations live in specially marked comments, transparent to the ordinary compiler, and use a superset of Java's expression syntax: a `requires` clause states a precondition, `ensures` a postcondition, `invariant` a class invariant, and `assignable` (the frame clause) bounds the locations a method may modify—indispensable for sound modular reasoning because it tells a verifier what a call cannot change. Postconditions may refer to the pre-state through `\old(e)`, to the return value through `\result`, and may quantify with `\forall` and `\exists`. Two properties are load-bearing. A JML contract is *machine-checkable*: unlike a natural-language comment it has a formal semantics against which an implementation can be judged mechanically, which qualifies it as an oracle rather than documentation. And a contract is *partial*: it constrains behaviour only so far as its clauses reach, which lets inference build a contract incrementally but makes a vacuous contract a first-class hazard—trivially satisfied, and so excusing incorrect code—that the thesis actively detects. The closest contemporary relative, Spec#, embeds contracts in C# with a verifying compiler [9]; JML's tooling maturity for Java makes it the practical substrate here.

2.2 Verification: OpenJML, extended static checking, and SMT

A machine-checkable contract is useful only if an implementation can be checked against it across all inputs. Deductive verification reduces "does this code satisfy this contract?" to a collection of *verification conditions* whose validity entails correctness, produced by a weakest-precondition computation over the method body [21, 44]. Loops are where the backward walk breaks down, since an unknown number of iterations cannot be enumerated; the remedy is a *loop invariant* the verifier assumes in place of unrolling. An invariant must be both *inductive*—true on entry and preserved by the body—and *strong enough* that, with the negated guard at exit, it entails the required obligation. Supplying invariants that meet both conditions is the largest source of the annotation burden that has historically deterred adoption, and a principal target of the inference surveyed in section 2.3.

Extended static checking (ESC), pioneered by ESC/Java [26], is the pragmatic engineering of this reduction: it sacrifices completeness for automation and modularity, analysing each method in

isolation while *assuming* the contracts of its callees rather than inlining their bodies. Modularity is what makes ESC scale and makes contract composition meaningful—a proof depends only on callee contracts, not implementations—but it bounds the guarantee: an ESC proof is only as sound as the contracts it assumes. OpenJML is the modern realisation of ESC for Java built on JML [14, 15]; this thesis adopts it as a trusted oracle rather than proposing a new prover. Beneath it is the Z3 SMT solver [18], deciding satisfiability of first-order formulas over the background theories—linear arithmetic, arrays, bit-vectors, uninterpreted functions—that Java constructs generate. OpenJML poses each condition in negated form: `unsat` certifies validity, while `sat` yields a *counterexample*, a concrete witness of violation that the integration chapter exploits as a signal far more informative than the bare fact that a test failed. Under modular checking, however, such a witness can be *spurious*, reflecting a too-weak invariant rather than a reachable defect, and the thesis does not overclaim what it establishes.

Precision about what a proof guarantees is essential. A successful ESC run proves the implementation satisfies its contract for *all* admitted inputs—a guarantee no finite test suite provides—but does *not* prove the contract captures what the developer wanted; that is the province of inference, not the verifier. The decidability limits must also be stated honestly. Verification is undecidable in general, and although Z3's quantifier-free fragments are decidable, the quantified formulas JML routinely generates are handled by heuristic instantiation and may return `unknown` or time out—in practice on nonlinear arithmetic, rich array reasoning, and deeply nested quantifiers. The corpora here exhibit exactly these modes, reported as solver *incompleteness and timeout*, not unsoundness in the inferred specifications. These limits are intrinsic to the condition-to-SMT architecture rather than artefacts of one tool, as the wider verifier ecosystem—Dafny [39], KeY [1], Frama-C [36], Verus [37], and the cvc5 backend [8]—confirms. A parallel synthesis tradition, counterexample-guided inductive synthesis and sketching [59, 60] standardised as syntax-guided synthesis [2], closes a generate–check–refine loop against a machine-checkable oracle—precisely the shape chapter 5 gives its synthesis–verification loop, with an LLM in place of enumerative search.

2.3 Specification inference and the oracle problem

The chief obstacle to deploying any contract-based machinery is the cost of *authoring* contracts. Specification inference attacks this bootstrap cost and is the technical core of the thesis's first claim, that specifications can be made cheap. The dynamic tradition observes a running program and reports the properties that held. Its canonical instrument, Daikon, records variable values at chosen points and reports the templated properties that survived every observed trace [23], establishing that a specification can be *recovered* from behaviour rather than authored from a blank page; specification mining similarly learns finite-state API protocols from traces [5]. The defining limitation is soundness: a mined property is only *likely* true, summarising sampled executions rather than all of them, so integrating it with a static checker is itself a research thread [49]. Such an invariant describes what the code *did*, not what it *should* do, and so cannot distinguish a faithful contract from an encoded bug.

The static tradition buys soundness by over-approximation. Abstract interpretation computes fixpoints over abstract domains so that every reported property provably holds [17]; Houdini takes the

complementary guess-and-check route, postulating candidate annotations and using an ESC verifier to refute the unprovable ones [25]—the historical antecedent of the verifier-in-the-loop discipline that recurs throughout this thesis. The price of soundness is real: rich domains do not always converge, and the decision procedure's limits bound what can be discharged. A learning-based strand poses invariant synthesis as learning from examples and verifier-supplied counterexamples [28, 58], and since 2023 the focus has shifted to LLMs prompted to emit candidate invariants a symbolic tool then checks, as in Loopy [35], or extended to full JML specifications with mutation-based fallback, as in SpecGen [43]; a separate sub-thread translates natural language directly into formal artefacts [16, 22, 55]. The common commitment the thesis endorses is that the generative model is never trusted alone but paired with a sound checker admitting only what it can prove.

The instrument carried through this thesis, the JML-Inferrer, occupies the static end of this space. It is a Java 21 tool over the JavaParser abstract syntax tree that emits JML—preconditions, postconditions, loop and class invariants, and `assignable` clauses—by heuristic pattern matching over syntax rather than complete program analysis: a null-check induces a non-nullness precondition, recognised loop shapes induce matching invariants and result postconditions, and numeric guards induce range and overflow preconditions. The heuristic basis is a deliberate trade, favouring recall of plausible, valid JML over the soundness guarantees of abstract interpretation, on the premise that the checker will reject any clause that does not hold—so an unsound guess costs a failed proof, not a false guarantee. The inferrer is thus static like Houdini but generative rather than refutational, over a rich JML vocabulary. Throughout, the thesis distinguishes a *verified* clause, one OpenJML has discharged, from a merely *well-formed* clause, one that is syntactically valid and scope-correct but unchecked, and never lets the distinction blur.

The dual of inference is the oracle problem. A test exercises a program; an *oracle* decides whether the observed behaviour is correct. Barr et al. [10] establish that while input generation is now largely automatable, the oracle remains the binding constraint, distinguishing *specified* oracles, derived from a formal description, from *derived* and *implicit* ones. Search-based generators sharpen the gap: lacking a specification, EvoSuite emits regression assertions that merely capture current behaviour [27] and Randoop flags only crashes and contract violations [52]. LLMs have accelerated this without resolving it: TestPilot repairs tests by re-prompting on failures [57], but recent analysis documents the hazard the thesis takes as its motivating defect—when the same model both implements the code and writes the asserting oracle, the oracle encodes the model's own mistaken expectations, yielding tests that pass by construction [42]. This thesis instead elevates an inferred, verified JML contract to a specified oracle discharged by OpenJML over all inputs, separating the authority that writes the contract from the agent that satisfies it.

2.4 LLM code generation and spec-driven lifecycles

The generators the verification machinery is meant to gate are LLMs and the agents built on them. Codex demonstrated function-level generation from docstrings and introduced the *pass@k* metric and HumanEval [13], with MBPP a complementary benchmark [6] and AlphaCode reaching median

human performance in contests [40]. These systems inverted the economics of classical synthesis—natural-language intent is cheap where formal specifications were not—but did so by abandoning the closed loop: correctness is *estimated* against a finite, model-adjacent test suite rather than proved. That estimate is fragile in two ways. Test suites are frequently too weak to expose plausible-but-wrong solutions, as EvalPlus showed by lowering reported pass rates through added tests [41]; and generation is prone to *hallucination*, the confident fabrication of non-existent APIs and subtly inconsistent logic [64]. The deeper problem is that generator and informal oracle share a model and training distribution, so verification is effectively self-grading, even as LLMs reshape the lifecycle wholesale [51].

The frontier has moved from isolated functions to *agents* over whole repositories. SWE-bench reframed evaluation around resolving real GitHub issues with project test suites as the oracle [34], and SWE-agent showed a purpose-built interface markedly improves navigation and editing [62]. The dominant oracle across this work remains test execution, inheriting the incompleteness of testing and, when tests are model-authored, the self-grading circularity at repository scale. Most directly related is the emerging family of *spec-driven*, AI-native lifecycles, which reorganise the process so a specification, not code, is the primary human-authored artefact—consonant with section 2.1, that when an agent regenerates the implementation cheaply the durable asset is the specification. GitHub's Spec Kit formalises a specify–plan–tasks–implement workflow anchored by a project "constitution" [29]; Amazon's AI-Driven Development Lifecycle restructures the lifecycle into inception, construction, and operations phases with quality gates and a brownfield reverse-engineering step [3]; and the Kiro IDE places a requirements-to-design-to-tasks workflow at the editor's centre [4]. These efforts correctly identify the economic inversion but share three structural defects the integration chapter remedies. The specification is *natural language*, so the specification-to-code step is a compilation with no checker relating generated code to stated intent. The loop is *open*: generation is followed by review-by-eye, not verification. And where verification appears at all it is *self-grading* test execution. A natural-language artefact is, besides, too imprecise to support automated reasoning over change—these lifecycles cannot compute which dependents a modification breaks without re-running anything.

2.5 Version compatibility as an implicit-contract problem

Modern software is assembled from libraries, and the contract between a library and its clients is communicated by a version number under semantic versioning: a major increment signals an incompatible change, a minor one backward-compatible additions, a patch a backward-compatible fix [53]. The promise is honoured unevenly. Raemaekers et al. [54] found roughly a third of Maven Central releases introduce a breaking change, frequently without the major increment; Ochoa et al. [50] replicated this over a far larger corpus, reporting higher compliance but noting most breaking changes touch code no client uses; and Decan et al. [19] characterise backward-incompatible updates as a structural hazard across packaging ecosystems. The version string is thus a weak proxy for what developers actually care about—whether *their* call sites still behave correctly.

Signature-diffing tools such as APIDiff compute deltas of type-, method-, and field-level changes

[11], and large studies catalogue such changes and their causes [12, 61], but share a ceiling: they detect changes to the *shape* of an interface, not its *behaviour*. A method whose signature is untouched but whose contract has silently shifted—an off-by-one boundary, a newly thrown exception—is invisible. This motivates work on *behavioural* incompatibility, the closest prior art to the compatibility chapters: Mostafa et al. [48] showed such signature-preserving changes are common and disproportionately damaging precisely because no diffing tool flags them, Jezek and Dietrich [33] that detection rates vary widely across tools, and Jayasuriya et al. [32] quantified breaks detectable only when client tests run. The recurring move is to recover behaviour *dynamically*—through client tests or mined invariants—which observes sampled inputs and so confirms an incompatibility but never its absence; a recent LLM direction *repairs* breaks once detected [31] but presupposes detection and guarantees nothing about the repair. The decisive gap is that compatibility is treated as an *ex post*, sampled, largely syntactic property; none of these strands answers the question posed at upgrade time—which of my call sites does this change break, and why—with a proof obligation rather than a failed test.

2.6 Positioning: the gap this thesis addresses

Each line of work optimises one edge of the problem in isolation. The verification community possesses a machine-checkable contract layer [15, 39] but presupposes a human authors the contract. The inference tradition recovers contracts cheaply [23, 25] but treats the inferred property as a terminal deliverable that describes what the code does, not what it should do. The LLM and agentic literature solved the authoring-cost problem that crippled classical synthesis [13, 62] but in doing so discarded the machine-checkable contract and the closed loop that made synthesis trustworthy, its spec-driven lifecycles institutionalising three defects—natural-language specs, an open loop, and self-grading verification [3, 29, 42]. And the behavioural-compatibility literature measures incompatibilities after the fact [31, 50] but does not derive a change's blast radius from a verified interface without running a client.

No prior line combines these capabilities, and it is the conjunction, not any single ingredient, that is novel. This thesis differs by being static, syntax-driven, and verification-disciplined: contracts are inferred from source without test inputs or traces, deterministically from the syntax, and every clause is held to OpenJML discharge or reported as a solver limit rather than shipped as an approximation. It is *compositional*: because contracts compose along the caller/callee obligation structure of design by contract, a callee-spec change recomputes each caller's proof obligations and flags behaviourally broken call sites without execution, making an internal refactor and a third-party version bump the same static blast-radius operation (chapter 4). And it is *integration-oriented*: the inferred, verified contract is interposed as the pivot of an AI-native lifecycle—the input an agent builds against and the oracle its output is verified against over all inputs—authored by an authority that never grades its own output (chapter 5). Chapter 3 establishes that such cheap, partial inference is nonetheless *useful*, measurably improving LLM-generated tests. The remaining chapters develop these capabilities in turn, reusing the inference and verification machinery established here as trusted components, and returning in chapters 6 and 7 to the limits that bound them—above all the boundary between verified

and merely well-formed specifications.

Chapter 3

The JML-Inferrer: Heuristic Static Inference

The thesis statement of chapter 1 rests on a premise that must be established before anything else can be built upon it: that specifications can be recovered from code cheaply enough to be worth having, and that once recovered they are good enough to be useful. This chapter discharges that premise. It presents the JML-Inferrer, the single instrument on which the empirical programme depends, and reports the two results that together answer RQ1: that the inferred contracts are of demonstrable *fidelity* — an overwhelming majority are formally consistent with the code they describe — and that they deliver measurable *downstream utility*, improving the unit tests a large language model writes from them. The conceptual foundations of design by contract, JML, and extended static checking are laid in chapter 2; this chapter takes them as given and concentrates on the instrument and its evaluation.

3.1 Architecture

The JML-Inferrer is a static analysis tool, implemented in Java 21 over the JavaParser abstract syntax tree (AST) library, that reads Java source and emits JML contracts as annotations inserted in place. The pipeline has three stages. A parsing stage turns each source file into an AST configured for the Java 21 language level. An inference stage traverses that tree with the visitor pattern, matching heuristic patterns against syntactic shapes and accumulating candidate clauses per method. A formatting stage renders the accumulated clauses as JML comment blocks and writes them back above their methods, preserving the original source otherwise untouched. Files are processed recursively over a directory tree, so the tool applies to a codebase exactly as it sits in a repository.

Two design commitments follow the instrument throughout and are worth stating at the outset because the evaluation depends on them. The first is that inference is purely static: it requires no test inputs, no instrumentation, and no execution traces, in deliberate contrast to the dynamic-detector lineage of Daikon, which generalises likely invariants from observed runs [23]. Trading the dynamic detector's ability to discover numeric relationships from data for the ability to run on any compilable code without a harness is the exchange that makes the tool applicable at scale, including to legacy code for which no adequate test suite exists. The second commitment is determinism: because every heuristic is a function of the syntax tree alone, the same input always yields the same contract. This

is not merely tidy engineering. It is what allows the downstream study of section 3.4 to treat the inferred specification as a controlled variable rather than a sampled one, and it is the sharpest point of contrast with prompting a language model to emit JML directly, an approach that returns non-identical contracts on identical inputs.

3.1.1 The contracts the tool infers

The inferrer emits up to five kinds of JML clause, spanning the vocabulary of design by contract [38, 45]. `requires` clauses state preconditions on the entry state; `ensures` clauses state postconditions relating the result and the final state to the entry state; `assignable` clauses state the frame, naming the locations a method may modify; `loop_invariant` clauses annotate loops; and class-level invariants state properties that must hold of an object between method calls. A single method typically receives a small block combining a guard precondition, one or more postconditions, and a frame clause; looping methods additionally receive invariants.

Postconditions carry the subtleties that most exercise the heuristics. A field that a method modifies must be referred to in its entry-state value under `\old`, so that `this.value == \old(this.value)+ operand.intValue()` expresses an increment rather than an impossible fixed point. Branching is preserved at clause granularity rather than collapsed: a method that returns one value on one path and another on a second yields two guarded postconditions, each pairing a return value with the condition that reaches it, of the form `cond ==> \result == e`. This decomposition is not only more faithful; it is, as section 3.4 shows, one of the mechanisms by which a downstream consumer can pick off a contract one clause at a time.

3.1.2 Heuristic families organised by WP and SP

The heuristics are organised by the weakest-precondition and strongest-postcondition correspondences developed in chapter 2, which supply the intuition for what a given syntactic shape licenses. Preconditions are recovered in the weakest-precondition direction: a guard-and-throw idiom near the top of a method body — `if (p == null)throw ...`, or a range check that rejects out-of-bounds indices — is read as an obligation the caller must have discharged, and is lifted into a `requires` clause. Postconditions are recovered in the strongest-postcondition direction: return statements and field assignments are propagated forward into `ensures` clauses, with pre-state field references wrapped in `\old` and path conditions attached where the value is reached conditionally. A family of loop heuristics — bound-based invariants for counter loops, accumulator-pattern detection, monotonicity inference from update expressions, and bounded symbolic unrolling as a fallback — supplies `loop_invariant` clauses, emitting a conservative weak invariant only when all of these fail. Method calls are handled in two passes: the first analyses methods in dependency order and caches each inferred summary, and the second propagates those summaries across call sites, resolving calls into the standard library against a curated specification database and treating calls into unannotated code as uninterpreted.

Throughout, heuristics that would be unsound under Java's two's-complement integer semantics are gated by type. A multiplication `x * x` or an `Math.abs(x)` does not license `\result >= 0` on an `int`

or `long`, because either can overflow to a negative value; the non-negativity clause is emitted only for floating-point return types, where it holds. Deliberately foregoing symbolic execution and full predicate-transformer computation — both of which would require an SMT decision procedure and a complete operational semantics for Java — is what keeps the inferrer scalable. Pattern matching trades formal completeness for breadth of applicability, the established bargain of industrial static analysis, and the resulting partiality is not hidden: no claim is made that a recovered contract is complete, only that it is sound, useful, and verifiable.

3.2 The verification discipline

What keeps heuristic inference honest is that every candidate contract is put to a verifier. Inferred annotations are checked by OpenJML's Extended Static Checker (ESC) [15], which discharges the resulting proof obligations through the Z3 SMT solver [18] without executing the program. A candidate clause that ESC cannot discharge is treated not as an acceptable approximation to be shipped but as a defect in the inference to be fixed. This discipline inverts the usual posture of heuristic tools: rather than emitting plausible output and leaving validation to the user, the inferrer is developed against the verifier, and a heuristic earns its place in the repertoire only once the verifier has discharged the clause it produces on a probe example. A containerised toolchain makes this reproducible, and an end-to-end regression suite of several hundred verification tests gates every change to the engine against the formal oracle.

The thesis is careful never to blur two statuses that this discipline makes distinct. A *verified* specification is one that ESC has discharged against the implementation: the checker has established the postcondition, invariant, or frame clause as a proof obligation from the body under the assumed precondition, or, for a precondition, has found the clause well-formed and its evaluation defined. A merely *well-formed* specification is syntactically valid and scope-correct JML that has not been through, or has not survived, that check. The distinction has teeth: some inferred clauses involve JML constructs the verifier cannot yet decide, and these are retained as well-formed oracle information without being counted among the verified. The remainder of the thesis preserves this boundary wherever the two might be confused, and chapter 6 returns to it as a threat to validity.

A specific obstacle in the verification back-end merited a fix rather than a work-around. Loop-accumulator postconditions are naturally expressed with the JML quantifiers `\sum`, `\product`, and `\num_of`, but the stock OpenJML build reports these as *not yet supported*, which would leave an entire class of accumulator contracts undischargeable irrespective of their truth. A forked OpenJML build emits SMT-LIB `define-fun-rec` definitions for these quantifiers, giving the solver a recursive definition to reason with and unblocking the accumulator-loop arithmetic that would otherwise stall. The fork is bundled with the toolchain so the validation pipeline remains reproducible. The strictness configuration is held fixed — operational integer semantics with overflow checks in code, mathematical integers in specifications, arithmetic exceptions treated as failures, and unannotated references treated as nullable — so that any change in the discharge outcome is attributable to the inference and not to a shifting oracle.

3.3 Inference fidelity

The first half of RQ1 asks how faithful the inferred contracts are. Fidelity is reported on two independent axes measured over a corpus of 312 methods drawn from eleven classes of Apache Commons Lang, a mature and diverse Java library whose preponderance of deterministic pure functions suits it to this kind of study. The first axis is *coverage*: the proportion of methods for which a non-trivial contract is emitted. The inferrer emits at least one non-trivial precondition or postcondition for 309 of the 312 methods, a coverage rate of 99.0%. The three uncovered methods are overloads that delegate to a standard-library operation for which no specification-database entry was available, whereupon the engine emits an empty contract rather than guess — the conservative limit of interprocedural summarisation rather than a heuristic failure. For comparison, a documentation-based baseline that recovers exception preconditions from Javadoc covers under half the same corpus, and prompting a language model to emit JML covers every method but with only 72.4% run-to-run consistency; the static inferrer, deterministic by construction, is perfectly consistent.

The second axis is *discharge*: the proportion of emitted clauses that ESC can verify against the implementation. Over all 985 inferred clauses, the checker discharges 92.9%, broken down by clause type in table 3.1. Two readings guard the figure. First, discharge is a soundness measure, not a completeness one: a vacuous clause would discharge too, so the number should be read as "no clause was found inconsistent with the implementation", which is why the coverage and structural-strength measures are reported alongside it. Second, discharge semantics differ by clause type — for preconditions the clause is assumed at entry and reported as failing only when ill-formed, so its column is a well-formedness check rather than a proof of necessity, whereas postconditions, invariants, and frame clauses are genuine proof obligations. The clauses that do not discharge separate cleanly into a small *flagged* set, routed to a review queue and excluded from any downstream use, and an *unsupported* set involving constructs the solver cannot yet decide. The residual non-discharge concentrates in a few identifiable patterns — recursive multiplicative growth, quantified invariants over mutable arrays, and the way OpenJML translates for-each loops — that are properties of the verification technology rather than of the heuristics, and that delimit what can currently be certified rather than what can be emitted.

Table 3.1: OpenJML ESC discharge over all inferred clauses on the 312-method corpus. *Discharged* clauses are verified against the implementation; *flagged* clauses fail discharge and are surfaced for review; *unsupported* clauses involve constructs the checker cannot yet decide.

Clause type	**Inferred**	**Discharged**	**Flagged**	**Unsupported**
Preconditions	312	296 (94.9%)	11 (3.5%)	5 (1.6%)
Postconditions	312	268 (85.9%)	22 (7.1%)	22 (7.1%)
Loop invariants	53	43 (81.1%)	4 (7.5%)	6 (11.3%)
Assignable	308	308 (100.0%)	0 (0.0%)	0 (0.0%)
Overall	**985**	**915 (92.9%)**	**37 (3.8%)**	**33 (3.4%)**

One caveat on fidelity carries through the thesis. The oracle against which clauses are discharged is the implementation, not the documented intent: ESC confirms that a clause is consistent with the code it was inferred from, not that it matches the abstract API contract. For a method whose intended

contract is strictly weaker than its implementation, the inferred clause will be sound against the body yet over-specified against the interface. This is intrinsic to code-derived inference and cannot be detected by a discharge oracle that is consistent with the over-specification by construction; chapter 6 records it, and chapter 4 shows how a second analysis pass recovers well-formed preconditions that single-pass heuristics miss.

3.4 Downstream utility: specifications as oracles

The second half of RQ1 asks whether these contracts are useful. The concern is not academic: a specification that is faithful but inert would not vindicate the thesis. The chosen demonstration is unit-test generation by a large language model, singled out not because it is the most important use of inferred specifications — later chapters pursue compositional verification and agentic integration — but because its effect can be measured cleanly. Language models are fluent at the mechanical part of a test, the prefix that drives a program into a state, but weak at the oracle, the assertion of what should then be true, because the oracle requires knowing what the method is *supposed* to do. A signature supplies no such knowledge and a body supplies too much, tempting the model to encode the implementation's actual behaviour, bugs included. A specification is precisely the independent statement of intent that both lack.

The study holds the model fixed and varies only the material supplied alongside an otherwise identical method signature, over four phases forming an ordered ladder of information density. P1 supplies the signature alone. P2 adds a fixed, method-agnostic checklist of edge-case categories — the control for the rival hypothesis that any structured guidance, rather than specifications in particular, raises test quality. P3 adds the inferred JML contract for the method, used without modification: the treatment condition. P4 substitutes the full source body, an upper bound available when the model can read the implementation directly. Each phase was run five times per method to control for sampling non-determinism, and the generated tests were evaluated by a fully automated pipeline that compiled them, ran them against the reference implementation, and computed a mutation score under PIT. Mutation score is the headline metric because it is the available objective proxy for oracle strength: a suite that kills more mutants is one whose assertions distinguish more behavioural deviations.

table 3.2 reports the corpus means. The scores rise monotonically along the information ladder, from 27.5% at P1 to 94.8% at P4. The principal contrast is P1 against P3: supplying the inferred contract raises the mutation score by 54.3 percentage points, a very large effect. Read against the full-source upper bound, P3 captures roughly $(81.8 - 27.5)/(94.8 - 27.5) \approx 81\%$ of the achievable gain from signature-only to full-source generation, and does so within an input an order of magnitude smaller — five to twelve lines of annotation against the thirty to a hundred and fifty lines of body that P4 supplies. The residual gap to P4 is largely the answer-copying that source code affords and a contract deliberately withholds: P3 exposes the intended behaviour without the implementation that may deviate from it.

The most instructive result is the comparison of P3 with the guidance control P2. The edge-case checklist yields the highest test count of any phase, 38.0 tests per method, yet its mutation score is

Table 3.2: Test-generation results by phase, as per-method means across the 312-method corpus. *Tests* is the average number of generated tests; *Mutation* is the PIT mutation score. Scores rise monotonically with information density, yet P3 attains a higher mutation score than P2 while generating fewer tests.

Phase	Input supplied with signature	Tests	Mutation %
P1	Signature only	9.0	27.5
P2	Signature + edge-case guidance	38.0	68.2
P3	Signature + inferred specification	33.5	81.8
P4	Full source body	54.7	94.8

13.6 percentage points *below* P3, which attains 81.8% while generating around 12% fewer tests. The divergence is the cleanest single piece of evidence for the thesis premise. Three patterns account for it: the checklist prompts near-duplicate cases that inflate test count without expanding the set of killed mutants; in the absence of an oracle its tests fall back on weak assertions such as "is not null" or "does not throw" that cannot distinguish a mutant which changes a return value; and P3's tests concentrate on individual contract clauses, giving a higher mutant-kill density per test. What distinguishes the treatment from the control is not that the model is told to be thorough but that it is told *what the method should do*. Test quantity is therefore a poor proxy for oracle quality, and the benefit being measured is specifications in particular, not structured prompting in general — had the study reported test count alone it would have concluded, wrongly, that generic guidance was the strongest intervention.

The benefit is uneven across method kinds in a way that confirms the mechanism. Control-flow methods gain most, because a guarded postcondition supplies an oracle for each branch that a signature leaves opaque, and utility methods gain substantially from preconditions that capture their input constraints; accessors gain least, not because a contract adds nothing but because their signatures already pin down their behaviour. Within the looping methods, those for which a non-trivial invariant was inferred outscore those left with a weak invariant, so the downstream benefit tracks the strength of the inferred specification. The right predictor of gain is the gap a contract fills over the signature, not its strength in isolation — and the two ends of the pipeline are coupled in exactly the direction the thesis predicts: where the inferrer produces a strong contract the model produces strong tests.

3.5 Summary

This chapter has presented the JML-Inferrer and answered RQ1 on both axes. On fidelity, the tool emits a non-trivial contract for 99.0% of a 312-method corpus, and OpenJML's Extended Static Checker discharges 92.9% of the resulting clauses against the implementation, with the residual failures traced to identifiable limits of the verification technology rather than to unsound inference. On utility, a controlled four-phase study shows that supplying the inferred contract to a language model raises the mutation score of its generated tests by 54.3 percentage points over the signature-only baseline, capturing roughly four-fifths of the achievable gain in an order of magnitude smaller input, and — most tellingly — outperforming a generic edge-case checklist that generates more tests but

weaker oracles. Automatically inferred, heuristic, partial specifications are thus both faithful and useful. That conclusion is the foundation the rest of the thesis builds on: chapter 4 shows how a second, weakest-precondition pass over the call graph strictly extends this inference and turns callee contracts into a basis for version-compatibility reasoning, and chapter 5 places inferred contracts at the centre of an AI-native development pipeline as the machine-checkable spine its verification loop requires.

Chapter 4

Compositional Specifications for Version Compatibility

4.1 From a local pass to a compositional one

The inference instrument of chapter 3 reasons about one method at a time. Its heuristics read the syntax of a single method body—a parameter dereferenced without a guard, an array indexed by a loop variable, an early `throw` that validates an argument—and emit a contract for that method alone. This locality is what makes the instrument cheap and deterministic, but it is also its ceiling. A method's true precondition is rarely a property of its own body only; it is inherited from the methods it calls. When a caller invokes a callee whose contract requires something of its arguments, that requirement becomes, after substituting the actual arguments, an obligation on the caller. Recovering such inherited obligations is the business of *compositional*, rather than local, inference.

The inference instrument already performs a limited form of this cross-method reasoning. During its single local pass it consults an `InterproceduralAnalyzer`: when it infers a method `m` that contains a call `n(args)`, it looks up `n`'s cached contract, substitutes the formal parameters of `n` with the actual arguments at the call site, and appends the substituted preconditions to `m`'s set. This is *single-pass propagation*: each method is visited once, each callee is read once, and its contract is pulled into the caller atomically. It is sound for the simple case—an obligation the callee requires *unconditionally* must hold wherever the callee is called—and it is fast and order-independent provided callees are inferred before callers.

The question this chapter answers, and the question that anchors the thesis's second contribution, is whether that single pass captures everything a genuinely compositional analysis would, or whether a second, deliberately compositional pass recovers obligations the first structurally cannot express. The answer matters twice over. If a second pass adds nothing, the simpler architecture is vindicated and the engineering cost of a second pass is unjustified. If it adds a great deal, then the instrument of chapter 3 was leaving recoverable specifications on the table, and—more consequentially—the same machinery that lifts a callee's contract into its caller is exactly what lets a *change* to a callee's contract be propagated across a call graph, which is the mechanism on which behavioural version-compatibility reasoning rests (section 4.6).

4.2 The weakest-precondition intuition

Where the single pass is blind is at call sites that are not reached unconditionally. Dijkstra's weakest-precondition calculus [21], resting on the axiomatic foundations of Hoare [30], gives the correct account: to compute what must hold at a method's entry, one walks the body backwards, accumulating the guard context that governs each program point. A statement reached only when a condition g holds contributes not its raw obligation p but the implication g $\Rightarrow$ p. A small example makes the gap visible. Consider a callee with an inferred, verified precondition and a caller that invokes it on one branch only:

```
//@ requires divisor != 0;
static int safeDivide(int a, int divisor) { return a / divisor; }

static int scale(int value, int factor) {
    if (factor != 0) {
        return safeDivide(value, factor);
    }
    return value;
}
```

At the call site `safeDivide(value, factor)`, substitution yields the clause `factor != 0`. The single pass now faces a dilemma with no correct resolution. Conjoining `factor != 0` unconditionally into `scale`'s precondition would forbid the very call `scale(value, 0)` that the method explicitly and correctly handles on its other branch; skipping the clause to avoid that error loses the obligation entirely on the path where the division does fire. A compositional walk resolves the dilemma by lifting the substituted clause through the enclosing guard, producing the branch-conditional clause `factor != 0` $\Rightarrow$ `factor != 0`. Here the guard happens to entail the obligation, so the lifted clause is a tautology and the example is deliberately trivial; in real code the guard and the callee's requirement differ, and a call to a method requiring a non-empty list, guarded by a size check, lifts to `size > 0` $\Rightarrow$ `list.size() > 0`—a clause that carries genuine information the single pass cannot state in any form.

A second structural class is equally out of reach. When a caller invokes `x.n()` on a receiver whose declared type has several implementations of `n`, runtime dispatch may select any of them, each with its own inferred precondition. Single-pass propagation resolves the call to one nominal callee and sees only that one. A compositional pass, aware of the class hierarchy, enumerates the candidates and emits the disjunction `(`p_1 `||` ... `||` p_k`)` of their substituted preconditions. This is a deliberate *weakening* of the caller's obligation rather than a sound safety guarantee—the heuristically inferred per-override contracts need not respect behavioural subtyping [48], so the strictly sound conjunction is frequently spuriously strong—but it is a shape a single nominal resolution cannot produce at all.

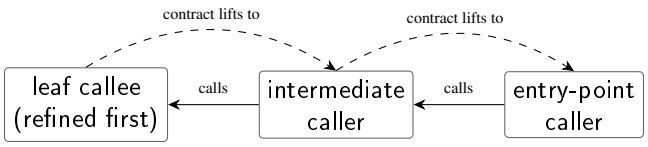

Figure 4.1: Refinement direction. The driver visits strongly-connected components of the call graph in reverse topological order, so each callee is refined before any of its callers; a callee's refined contract then lifts into the caller as a substituted, guard-conditioned obligation.

4.3 The compositional refinement pass

The realised design is a lifting step layered over the existing local result, not a full weakest-precondition transformer. A complete transformer would require a decision procedure and a precise operational semantics for every Java construct, which is the province of full verifiers such as OpenJML's static checker [15, 38], not of a heuristic inferrer. The pass instead takes as input the populated specification cache from chapter 3, a call graph over the same methods, and the abstract syntax tree of each declaration, and it mutates the cache in place.

A driver iterates the strongly-connected components of the call graph in reverse topological order, so that callees are refined before their callers (fig. 4.1). A component with no recursion is refined in a single sweep; a mutually recursive component is refined repeatedly until no method gains a new clause, or until a configurable cap (sixteen iterations) bounds the runtime on pathological graphs. The two-tier treatment is forced by the graph rather than chosen for convenience: on an acyclic graph one reverse-topological sweep already refines every method against fully refined callees, but a cycle admits no such order, so a method first visited against a still-stale callee must be revisited once that callee is itself refined. The iteration terminates because each method's clause set only grows and is bounded above by the finitely many substituted callee clauses reachable in the component.

For each method the refiner walks every call expression in the body and, at each call site, resolves the callee, substitutes the actual arguments for the callee's formal parameters, lifts the substituted clause through the conjunction of any enclosing `if` or ternary guards (with `else` branches contributing the negated guard), detects polymorphic dispatch and emits a disjunction over candidates where it occurs, and conjoins the result into the caller's set if it is not already present. Substitution is textual with word boundaries, so a formal parameter `s` does not clobber a substring of `this.size`. Two soundness gates reject unsafe substitutions outright: a call whose arguments or whose enclosing guards are side-effecting (an increment, a decrement) is skipped, because substituting an expression whose evaluation changes state does not preserve meaning. Postconditions are propagated by the same machinery but gated on a termination flag—a non-terminating callee's postcondition describes a state established only if the callee returns, so propagating it unconditionally would be unsound.

4.4 Entry-state well-formedness

A lifted clause is only admissible if it can be *stated* at the caller's entry. A JML `requires` clause may reference only entry-state names—the method's parameters and the fields visible to it—and never a

body-local such as a loop index or a temporary, because a precondition is evaluated before the body runs and body-locals do not yet exist. Two routes threaten this discipline. Lifting a guard that tests a body-local (`if (i < n) ...`, where `i` is a loop variable) would drag `i` into a `requires` clause; and substituting a caller-local argument into a callee's precondition would do the same by the other route. Either produces a clause that is syntactically ill-formed as a precondition, no matter how faithful it is to the program's meaning.

The pass therefore checks every candidate clause against the caller's entry scope before emitting it, discarding any clause that mentions a name not in scope at entry. This filter—realised as the `EntryScopeFilter` component—is not cosmetic. Enforcing it materially lowered the recovered count: an earlier, unfiltered version of the analysis reported on the order of forty-five thousand new preconditions, but many of those referenced body-locals and were not legitimate entry-state obligations at all. With the filter enforced, the pass yields roughly **18,854 additional well-formed preconditions** across the study corpus (section 4.5), and a post-hoc re-scan confirms that none of the emitted clauses references a body-local on any library. The corrected figure is the one this thesis reports and defends.

The distinction that governs the whole chapter must be stated plainly here. A *well-formed* clause is syntactically valid JML and scope-correct: it names only entry-state values and could, in principle, be handed to a verifier. A *verified* clause is one an SMT-backed checker has actually discharged [15, 18]. The compositional pass produces the former, not the latter. It is a structural transformation over already-cached contracts: given the callee specifications, whatever their semantic quality, it produces new caller clauses and confirms they are expressible at entry. It does not run OpenJML on them, and it makes no claim that the 18,854 clauses are individually sound against the code's intent—that is the inference instrument's responsibility, discharged by the verification discipline of chapter 3, not the compositional pass's. Throughout this chapter, therefore, the headline number is a count of *well-formed* preconditions, not verified ones, and the two are never conflated.

4.5 Measurement study

The pass was measured, not proved. The study asks how many additional well-formed preconditions the second pass recovers, over what fraction of the call graph, on real Java. The corpus is the source of five open-source libraries chosen to span four idiom families: Apache Commons Lang and Commons IO (imperative utility-class and I/O idioms), Commons Math (numerical algorithms), and jOOL and Vavr (higher-order, generics-heavy functional idioms). Commons Lang and Commons IO are shared with the studies of chapter 3, which keeps cross-chapter comparisons free of corpus-difference confounds. Combined, the corpus is 1,712 source files and 21,052 methods that carry a non-empty local baseline contract. For each library the harness parses every source file, builds the call graph, runs the local inference (including its single-pass interprocedural propagation) to snapshot the baseline, invokes the compositional pass, and diffs each method's refined contract against its baseline, counting and categorising the newly added clauses. Every measurement is deterministic given the inputs and is reported once.

Table 4.1: Effect of the compositional pass over the local baseline, per library. "Refined" methods gained at least one new clause; every emitted clause is well-formed (entry-state-expressible), not separately verified.

	Lang	**IO**	**Math**	**jOOL**	**Vavr**
Methods in cache	3,494	2,111	6,645	3,691	5,111
Methods refined	1,367	711	2,203	784	1,541
% of cache	39.1	33.7	33.2	21.2	30.2
New precondition clauses	2,632	1,329	8,966	4,262	1,665

The second pass strictly extends the baseline on every library (table 4.1). It adds at least one clause to between roughly 21 and 39 per cent of cached methods—median about a third—and in total recovers **18,854 new, well-formed precondition clauses** (together with a further body of new postcondition clauses) across the five libraries. The effect is a population-level one rather than the artefact of a few enormous methods: the mean number of new preconditions per refined method is modest, in the low single digits on every library, well below the per-library maxima, which are therefore deep-tail outliers. Crucially, this gap persists even though every library's baseline already invokes single-pass interprocedural propagation; the second pass discovers obligations the first cannot express, not obligations the first simply failed to compute.

The shape of the additions explains *why* the single pass falls short. Categorising each new precondition by its syntactic form, the two shapes that lie outside the single pass's reach by construction—branch-conditional implications (`guard` $\Rightarrow$ `obligation`) and disjunctions from polymorphic dispatch—are present on every library and together account for between two-fifths and three-fifths of new clauses on each. Branch-conditional clauses are the single largest shape on three of the five libraries, exceeding two-fifths on Commons Lang and on Vavr, and remain at least a third on the other two, where pervasive input validation instead pushes plain null checks to the plurality (the matrix and array arguments of Commons Math, the lambda pinch-points of jOOL). Disjunctive clauses, the polymorphic-dispatch signal, peak on Vavr, the library with the deepest interface-inheritance chains. Null and range obligations also appear, but these are shapes the single pass *would* have produced at an unguarded call site; the second pass adds them only because the call site was guarded by a condition the single pass would not lift. A syntactic vacuity test confirms that the branch-conditional counts are not inflated by tautologies of the `g` $\Rightarrow$ `g` form: such vacuous clauses are a negligible fraction of the branch-conditional total on every library.

The pass is cheap. Across all five libraries the refinement runs in around fifty seconds in total—on the order of a couple of milliseconds per method—an addition of seconds to a local inference that already takes minutes on libraries of this size. The fixpoint iteration never fired: under the analyser's signature-matching scheme all five call graphs are acyclic, so each method is refined exactly once and the sixteen-iteration cap is never approached. This is a statement about precondition propagation along the call graph specifically, and it does not generalise to every specification a compositional inferrer might produce; class invariants, whose inference is a greatest-fixpoint driven by shared-state coupling rather than by call edges, are a genuinely harder problem left to future work and revisited in chapter 6. Two caveats bound the headline. Callee resolution uses simple-name matching, which

rejects some overloaded call sites on arity mismatch, so unmatched sites can only ever add clauses the count omits; the reported figure is thus a *lower bound* on the true compositional gain. And clause equality is syntactic, so the count is an upper bound on the number of distinct *logical* obligations. Both caveats concern the count's precision, not the qualitative finding, which is robust: the single pass is not subsuming on any library measured.

4.6 Behavioural version compatibility

The mechanism that recovers a caller's obligations from a callee's contract is, turned around, a mechanism for propagating a *change* in a callee's contract to everything that depends on it. This is the pivot from measurement to application, and it is the concern that led the thesis from the outset: library and version compatibility.

Modern software is assembled from libraries, and the contract between a library and its clients is signalled by a version number under the convention of semantic versioning [53], which promises that a compatible release will not break existing callers. The convention is honoured unevenly: breaking changes are routinely shipped in releases whose version number declares them compatible [54], precisely because the contract being versioned is implicit. There is no machine-checkable statement of what a release guarantees, and hence no mechanical way to decide whether a new release still honours what the old one promised. The consequence falls hardest on exactly the systems whose behaviour is least understood—the legacy systems that most matter [24]—and it is amplified by any lifecycle that regenerates components against informal descriptions, where every regeneration is a potential silent breaking change.

Inferred, compositional contracts turn this asserted convention into a checkable property. Given two consecutive releases of a library, the instrument infers a contract for each method in each release; diffing the contracts method by method yields the set of methods whose behavioural interface has changed, and the compositional pass then propagates each change across the call graph to compute its *blast radius*—the set of callers whose own obligations shift as a consequence. A precondition that a release has *strengthened* is the sharp case: client code that satisfied the old requirement may now violate the new one, which is a breaking change in behaviour whether or not the version number admits it. The analysis is entirely static and run-free—no client is executed, no test is run—and so it applies to a release exactly as it sits in a repository.

A concrete version step demonstrates the mechanism operating at scale. Inferring contracts across two consecutive Commons Lang releases separated by roughly two years, and diffing them, isolates a tractable candidate-breaking set: of the 3,188 methods present in both releases, around 525 (some 16.5 per cent) have an inferred contract that differs—exactly the set a behavioural-compatibility checker should surface. Of these, on the order of 144 have a *strengthened* precondition, the class of change most likely to break an existing caller; a spot check finds a mixture of real precondition tightenings, such as added non-null guards on the instance fields of several classes, together with some inferrer noise. Two properties make this output useful even before each candidate is validated against the release notes. It is the right *size*—hundreds of candidates out of thousands of methods, small enough

for a human or a downstream tool to triage—and it is the right *shape*, classified into strengthened, weakened, and mixed changes on both the precondition and postcondition sides, so the class of each change is legible.

The reasoning is behavioural rather than syntactic, and this is the substance of the contribution. A textual diff of two releases reports that a line changed; it cannot say whether the change tightens what a caller must guarantee. A contract diff, propagated compositionally, reports that a caller's obligation shifted and in which direction—the semantics of the change, not its surface. This is what it means to turn semantic versioning from a convention a maintainer asserts into a property a tool can check, and it is the mechanism on which the AI-native verification pipeline of chapter 5 relies to keep regenerated components compatible with the callers that depend upon them.

The claim must be stated at its true strength, and no stronger. What the study establishes is that the compositional pass recovers a large body of additional *well-formed*—not separately verified—preconditions, and that the same propagation machinery yields a candidate-breaking set of the right size and shape to make behavioural-compatibility triage tractable on a real version step. It does not establish that every candidate is a true breaking change, nor that the recovered clauses are individually verified; validating the candidate set against release notes, and discharging the recovered clauses through OpenJML at corpus scale, are the natural next steps set out in chapter 7. The contribution is a measured extension of the inference instrument and a working static analysis for behavioural compatibility, presented as evidence rather than as a proof of correctness.

Chapter 5

Integrating Inferred Specifications into the AI-Native Lifecycle

The two preceding chapters established that specifications can be inferred cheaply and put to use. Chapter 3 showed that heuristic inference recovers sound, verifiable contracts from Java source and that supplying them to a language model measurably improves the tests it generates; chapter 4 showed that a second, weakest-precondition pass strictly extends what single-pass inference can recover and, in doing so, turns behavioural version compatibility into a static analysis. Each treated an inferred contract as an end in itself—an oracle for test generation, a property to refine. This chapter draws the threads into a single lifecycle argument. It asks what a software development process would look like if it were reorganised around a machine-checkable contract as its durable source of truth, and answers with a verification-centric *reference architecture* in which that contract is the pivot between ambiguous human intent and precise machine verification. The instruments of the earlier chapters reappear here not as fresh contributions but as the trusted components from which the architecture is assembled. The conceptual background—program synthesis, the test-oracle problem, and behavioural version compatibility—is surveyed in chapter 2 and not repeated.

The motivation is an economic inversion already argued in chapter 1. When an agent can regenerate a conforming implementation from a description in seconds, the implementation ceases to be the treasured asset and becomes closer to a disposable build output; what becomes scarce and durable is the specification of intent against which any number of cheap regenerations are measured. The emerging agentic, specification-driven lifecycles—GitHub's Spec Kit [29], Amazon's AI-Driven Development Life Cycle [3], and the Kiro integrated development environment [4]—have noticed the inversion and place a "specification" at their centre, but the specification they organise around is natural language, which is not machine-checkable. Examined as a control system, these lifecycles exhibit three compounding defects: the spec-to-code step is an unverifiable compilation from an informal source; the loop is open, closed only by human review or a sampled test suite; and where automated acceptance exists it is circular, because the same model that writes the code also writes the oracle it is graded against [41, 42]. These are one missing component viewed from three angles: a *verification spine*. This chapter supplies it.

5.1 Enabling machinery: carriage and continuous integration

Before the architecture can be described, two prior results must be recalled as enablers, because a lifecycle organised around specifications presupposes that specifications can travel with the artefacts developers exchange and can be produced routinely without disrupting the team. Both were established as contributions in their own right; here they are compressed to the role they play in the larger pipeline.

5.1.1 Carriage: specifications that survive compilation and publication

A specification that cannot travel with the artefact it describes lives in a separate document that drifts out of date and is unavailable to the very consumers—verifiers, IDEs, test generators, and models reading an API—that would benefit from it. Java libraries ship as JAR files and REST endpoints expose only an HTTP signature, so source is the exception rather than the norm. The carriage mechanism resolves this by writing inferred JML [38] into runtime-retained class-file annotations via an ASM-based embedder, with a parallel OpenAPI 3.x extension family (`x-jml-*`) for service boundaries and a JML stub-file sidecar that OpenJML [15] reads natively for verification. The design descends from the design-by-contract observation that a contract is a property of the supplier as seen by a client who most often holds only the binary [45]. Empirically it round-trips losslessly: across six mature libraries totalling 39,268 methods, and across 22,881 real inferred specifications recovered from those libraries' source, every specification the harness embeds is recovered byte-for-byte, at a bytecode overhead of roughly 4.6–10.2% after format optimisation and at throughputs that comfortably support per-pull-request continuous integration. Embedded classes load correctly in JVMs that have never seen the annotation type, so a JAR can be enriched in place with no risk to consumers and no toolchain change. The single fact the lifecycle needs is that a contract produced by inference can be carried, at full fidelity, on the artefact a developer actually deploys.

5.1.2 Continuous integration: inference as a routine, fail-open step

For inference to change how software is built it must run where development happens—in the CI pipeline, on every change, without disrupting the merge train. Formal-method tooling is conventionally held to be too slow and too brittle for this, and three deliberate design choices overturn that calculus. First, *diff-only analysis with content-hash caching*: whole-codebase re-inference runs once per merge to the trunk, while per-pull-request work is restricted to changed methods, unchanged methods being served from cache. The cache key is a SHA-256 hash of the method source together with the canonical form of its callees' specifications and the inferrer version, so a callee's specification change invalidates precisely its callers and nothing else—the property that makes the cache friendly to the compositional analysis of chapter 4. Second, *fail-open semantics*: an inference crash, an OpenJML timeout, or a malformed clause is logged to the pull-request comment but never blocks the merge, so the pipeline is safe to deploy where formal methods have previously been resisted; strict gating is available only as an opt-in. Third, *persistent in-place reporting*: a reviewer sees one collapsible summary per pull request rather than a growing thread of comments. A working reference

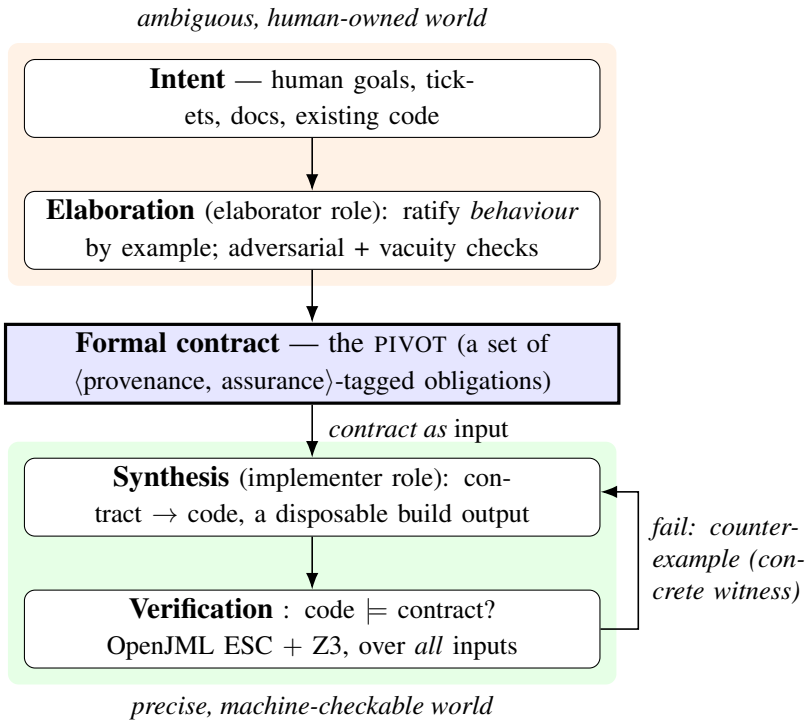

Figure 5.1: The reference pipeline. The formal contract is the pivot between the ambiguous, human-owned world and the precise, machine-checkable world. Elaboration converts intent into a contract under human ratification of behaviour; synthesis and verification form a counterexample-guided loop below the pivot; and the verifier never grades a contract it authored.

implementation—a command-line driver with `infer`, `verify`, `embed`, and `summary` subcommands, plus templates for GitHub Actions, GitLab CI, and Jenkins—is dogfooded against the inferrer's own repository, where a clean-cache run completes in roughly ninety seconds and a deliberately introduced syntax error is surfaced without blocking the merge. The controlled study of pipeline overhead across external repositories is designed but not yet run, a boundary chapter 6 revisits. The enabling fact is that inference can be a routine CI citizen: cheap, incremental, and unobtrusive enough to run as a matter of course.

5.2 A verification-centric reference pipeline

With carriage and CI in place, the lifecycle can be reorganised around a machine-checkable contract. The pipeline is offered as a reference architecture—a set of design principles, a layered model, and an end-to-end data flow—validated by instantiation rather than asserted as a proven methodology. Figure 5.1 shows its shape.

5.2.1 The contract as pivot, and its dual role

A machine-checkable formal contract sits at the centre of the architecture as the *pivot* between the informal activity of deciding what is wanted and the mechanically checkable activity of producing and certifying code. It is the single point at which informality is converted into formality and the

only artefact that crosses that boundary. Concretely the pivot is expressed in JML [38], whose pre/postcondition discipline descends from Hoare logic [30], Dijkstra's weakest-precondition calculus [21], and Meyer's design by contract [45], and discharged by OpenJML extended static checking [15] over Z3 [18].

The decisive property is that the contract plays two roles at once. It is the *input*—the precise intent the synthesis agent builds against—and it is the *oracle*—the artefact against which that agent's output is formally verified. This duality makes the pipeline self-consistent: there is exactly one statement of intent, and code is judged against the very artefact that produced it, which is precisely what the natural-language lifecycles cannot achieve, because their input has no formal semantics against which output could be checked. The historical obstacle to the dual role was the cost of authoring formal contracts; that obstacle is removed by the inference of chapter 3 and chapter 4, which supplies a draft automatically, so the human's task collapses from *authoring* formal text to *ratifying* behaviour.

5.2.2 A closed synthesis–verification loop

Below the pivot the ambiguous human world has been left behind: the contract is a precise object, and the agent's task is no longer to interpret intent but to satisfy an obligation. Here the open generate-then-eyeball cycle is replaced by a closed loop shaped like counterexample-guided inductive synthesis (CEGIS) [59, 60]. A synthesis agent proposes a body; the trusted verifier attempts to prove it satisfies the contract over all inputs; and any failure is returned to the agent not as a verdict but as a concrete counterexample—a fully instantiated pre-state in which the body cannot discharge the obligation—that drives the next attempt. The value lies entirely in the quality of that feedback. Self-improving code agents close their loop with a sampled test suite, which certifies only the finitely many inputs it happened to exercise and whose weakness as a correctness oracle is well documented [13, 41]; a failing test reports that one input misbehaved but gives no assurance that fixing it will not break another. A successful ESC proof, by contrast, certifies the obligation over the entire input domain, and a failed one hands the synthesiser a witnessed cause in the same logic the verifier reasons in. Because any method with a loop needs an inductive invariant that ESC cannot guess, the loop separates two hard problems: the agent contributes the algorithmic content, and the inferrer of chapter 3 supplies the discharging loop invariant—in the direction-aware, inductively preserved form ESC requires—for exactly the loop families it handles. When the solver neither proves nor refutes within its budget—Z3 is incomplete on multiplicative arithmetic, array theory, and richly quantified preconditions—the obligation is not dropped but *degraded* down a ladder from full proof to bounded checking to a generated test, with the level achieved recorded on the obligation rather than silently lowered. This graduation is what makes a verification-centric lifecycle realistic rather than aspirational.

5.2.3 Independent authority: a model must not grade its own work

Because the contract is the oracle, verification is meaningful only if the contract is not authored by the same agent that writes the code. If one model both proposes the contract and satisfies it, the verifier still soundly proves that the code meets the contract, but that proof certifies nothing about *intent*: a

shared misreading is encoded identically in contract and code, which then agree without exposing it. This is the self-grading circularity that undermines test-suite acceptance and, more acutely, model-driven oracle generation [42]. The *independent-authority constraint* therefore partitions the lifecycle into an *elaborator* role—maximally faithful, never permitted to see the implementation, and the owner of the ratified examples—and an *implementer* role, whose only task is to satisfy the contract handed to it. The verifier, reused unchanged, is authored by neither. The constraint's scope must be stated honestly: it removes self-grading, but it does not by itself decorrelate a shared misreading of intent across two roles of the same underlying model. That residual is broken only by human ratification, whose authority lies outside the model distribution. Ratification is made feasible by a discipline of *ratifying behaviour, not logic*: rather than confirm a page of quantified JML, the human is asked concrete questions in domain terms—"given an empty list, this contract throws rather than returns zero; is that what you want?"—and each confirmed answer becomes a machine-checkable pinned example the contract must henceforth satisfy. Every obligation carries a provenance tag recording its origin and an assurance tag recording the strongest verification level achieved, so that adherence to the constraint is auditable and the word "verified" stays honest: a reader can see which obligations are human-ratified, which are merely machine-inferred, and which were proved as opposed to merely tested.

5.2.4 Brownfield contract recovery by triangulation

Most software is not new, so in the common case no contract exists and one must be *recovered*. The difficulty is that the existing code is at once the best evidence of what the system does and the prime suspect, since any bug it contains is encoded in exactly that code; a contract inferred from code alone therefore describes the implementation faithfully, defects included. The recovery phase consequently triangulates three independent evidence streams. The *code*, via the inferrer of chapter 3 and chapter 4, answers what the system does; the *issue tracker* answers why, recording the defects and edge cases the behaviour has provoked; and the *documentation* answers declared intent. The reconciliation is deliberately not a merge that smooths away conflict. Its primary product is *disagreement detection*: the points at which the streams make incompatible claims are each a hypothesis that something is wrong—the code, the documentation, or the recorded understanding. A clause the code cannot satisfy but that a documentation- or ticket-derived stream asserts is direct evidence of a code-versus-intent gap. This reframing forces a reordering of the loop. In greenfield use it is prospective; the first brownfield pass is *retroactive*, resolving the recovered discrepancies and establishing a ratified baseline before the pipeline is permitted to author any change. Because a whole legacy system cannot be formally specified, recovery adopts a *frontier* strategy: it specifies only the module under change, the public API surface, and the modules named by active tickets, treating the remainder as governed by weak inherited-default contracts. Verification during this phase must never be read as a proof of correctness; when the contract is dominated by code-inferred clauses, verifying the code against it is nearly circular, establishing only that the implementation is self-consistent. Correctness relative to intent comes from the intent streams and, ultimately, from human ratification—never from the code stream alone.

5.2.5 Compositional change propagation as a lifecycle capability

The final capability promotes the compositional refinement of chapter 4 from an inference-time accuracy gain to a lifecycle operation. Because contracts compose along the call graph, a change to a callee's contract mechanically *recomputes* the proof obligations of every caller, and the callers whose obligations can no longer be discharged are exactly the callers the change breaks—identified by re-running the verifier, not the program. The analysis is directional in a way that mirrors behavioural subtyping: weakening a precondition or strengthening a postcondition is safe and flags no caller, whereas strengthening a precondition or weakening a postcondition is potentially breaking and yields a ranked, justified, per-call-site work list. The same machinery answers two questions the prevailing tooling treats as unrelated—"does this internal refactor break any call site?" and "does this dependency upgrade break any call site?"—because, at the level of contracts, an internal refactor and a third-party version bump are the same operation. This is the concrete link between the two strands of the thesis core: formal-methods adoption and library compatibility are served by one mechanism. It addresses a real gap, since semantic versioning is an unreliable compatibility signal [53, 54] and signature-level tools are blind to behavioural breaks that leave the signature untouched [48]. The carriage mechanism of section 5.1.1 is what makes the library side feasible without source, and the cache-backed CI of section 5.1.2 is what makes re-running the analysis on every upgrade economical. The soundness and the limits are those of the underlying instrument: a caller reported compatible is compatible over the whole specified domain, but a caller the solver cannot decide is reported as undetermined rather than silently passed, and because an inferred callee contract skews weak, a genuine break may be missed where the inferred contract failed to capture the guarantee the caller relied upon. The analysis is therefore sound in flagging breaks but not complete, and trustworthy only within the specified frontier.

5.3 Designed versus measured

A scruple must be stated plainly and held throughout. This pipeline is a *reference architecture synthesised from validated components*, not an empirically evaluated deployment. Its trusted core already exists and has been independently validated in the earlier chapters: the JML-Inferrer and its compositional pass that bootstrap draft contracts (chapter 3, chapter 4), the OpenJML fork with Z3 as the oracle that never grades itself, the artefact-embedding machinery of section 5.1.1, and the Dockerised CI pipeline of section 5.1.2. The only new software is untrusted orchestration—sequencing the phases and managing the counterexample loop—plus the model calls that draft and synthesise; none renders a verification verdict. The architecture has been exercised only in part: the synthesis–verification loop against fixed, known-dischargeable contracts on Apache Commons Lang, while contract generation from intent and conversational ratification are specified as a staged build plan with full validation deferred. The claim is accordingly one of *feasibility and realism*—that the spine can be assembled from parts that already work and made to surface real code-versus-intent discrepancies on a genuine three-stream corpus—not one of a measured effect at scale. The distinction between a *verified* obligation, which OpenJML has discharged, and a merely *well-formed* one, which is syntactically valid

and scope-correct but unproved, is never allowed to blur here: the compositional pass of chapter 4 contributes well-formed preconditions, and only those the verifier discharges are verified. Chapter 6 takes up the threats to this position in full, and chapter 7 restates the boundary between what has been built and what remains to be measured.

Chapter 6

Discussion and Threats to Validity

6.1 What the three results together establish

Taken singly, each of the preceding results answers a bounded question; taken together they trace a single arc, from inference through composition to integration. Chapter 3 establishes that specifications can be recovered cheaply—the JML-Inferrer recovers sound, verifiable contracts from unannotated Java by syntax-driven heuristics, and supplying those contracts to a language model measurably improves the unit tests it generates. Chapter 4 establishes that the same inferred contracts can be reasoned *about*: a compositional, weakest-precondition pass over the call graph strictly extends the local heuristics, and—run differentially across two versions of a dependency's contract—turns behavioural version-compatibility into a static, run-free analysis. Chapter 5 shows what those capabilities make possible once assembled: a reference architecture in which a machine-checkable contract is the load-bearing pivot between ambiguous human intent and precise machine verification, serving at once as the input an agent builds against and the oracle its output is checked against. The movement is from cheap specifications, to reasoning about compatibility, to specifications made load-bearing in a lifecycle.

The three research questions may now be assessed against that arc. **RQ1** asked how faithfully specifications can be inferred from unannotated code and whether the result is useful downstream; it is answered by chapter 3 and answered by *measurement*. Fidelity is anchored to formal verification rather than inspection—where the engine emits a contract and OpenJML [15] discharges it through Z3 [18], the contract is sound with respect to the code—a stronger guarantee than the likely-invariant detection of dynamic tools such as Daikon [23], and the downstream benefit is established by a controlled comparison rather than asserted. **RQ2** asked whether a compositional pass extends inference and whether it enables version-compatibility reasoning; it is answered by chapter 4, in part by measurement (the additional well-formed preconditions recovered on a large corpus) and in part by a design validated by instantiation (the differential blast-radius analysis). The link is not analogy but identity: the arrow that, run forwards within a version, strengthens a caller's specification is, run across versions, the edge that carries a breaking change from a library to its clients. **RQ3** asked whether inferred contracts can serve as the contract layer and verification loop of AI-native lifecycles; it is addressed by chapter 5 as a reference architecture that is *designed, not deployed*. The architecture is concrete

and its components are individually evidenced by the earlier chapters, but the end-to-end lifecycle is demonstrated for feasibility and coherence, not established by a controlled effect size. That distinction is maintained deliberately throughout, and the industrial pilot that would supply such a size is future work (chapter 7).

6.2 Threats to validity

The value of a candid thesis lies partly in its restraint, and several threats bound the claims above.

Corpus generalisability (external). The quantitative evaluation, and the compositional-reach measurement in particular, leans on Apache Commons Lang and similarly structured Java libraries. These are real, widely-depended-upon codebases rather than synthetic benchmarks, so the results are representative of an important class of production Java; they are *not* random samples of all Java, and the thesis does not claim that the reported figures transfer unchanged to arbitrary corpora. A library written in a markedly different idiom—heavy reflection, dynamic proxies, concurrency-dominated control flow—may expose patterns the heuristics do not recognise and obligations the solver cannot discharge. Multi-corpus replication is the obvious next step.

Heuristic partiality (internal). The inference engine recognises the patterns it was built to recognise and is silent elsewhere, so its inferred contracts are sound *fragments*, not complete specifications. This biases the population of contracts toward the tractable and could inflate apparent fidelity if the easy cases are also the verifiable ones. The mitigation is to report fidelity as a rate conditioned on emission and verification, and to record the engine's silence honestly rather than count it as success: the thesis claims soundness where a contract is emitted and verified, never coverage of all behaviours.

SMT boundaries (conclusion). The verification spine rests on Z3, and the underlying logic is undecidable. The programme repeatedly encounters timeouts on multiplicative arithmetic, quantified-array-theory queries, and rich preconditions; where the solver cannot decide, the pipeline neither confirms nor refutes. An honest account records such obligations as *unverified*, not as results the tooling did not deliver. Operationally the configuration is tuned to maximise the fraction discharged within budget, and the fail-open, sound-in-alarm posture ensures that an undecided obligation degrades to "unverified" or "unflagged" rather than to a false guarantee. For the compatibility analysis this makes the reading asymmetric and must be stated as such: an alarm is trustworthy, but silence is not a guarantee of compatibility, because a break may be missed when a callee contract is inferred too weakly to capture the depended-upon behaviour, or when a timeout leaves the relevant obligation unproved.

Verified versus well-formed (construct). The compositional preconditions recovered in chapter 4—roughly 18,854 on the corpus studied—are *well-formed*: syntactically valid and scope-correct, admitted only when their entry-state scope is satisfiable. They are *not* separately discharged by extended

static checking, and the thesis never blurs the two. A verified specification is one the checker has proved against the code; a well-formed one has passed a weaker, structural filter. Reporting the well-formedness-filtered count as the headline figure is itself part of the mitigation, since it excludes vacuous contracts—an inference emitting `ensures true` is trivially verifiable yet says nothing—from being counted as wins.

Construct validity of the utility measure. The downstream-utility study of chapter 3 measures test quality by mutation score, a proxy for the oracle strength of a generated suite [10] rather than a direct measurement of correctness caught in the field. A proxy can diverge from the construct it stands for. The mitigation is that mutation score is an *external* effect a vacuous contract cannot produce, giving a check on construct validity independent of the contract's internal form. A related internal threat is run-to-run variance in the language model: its output is stochastic, so a test-generation result depends on a model whose behaviour varies between runs. This is mitigated by the paired design—the same prompts with and without the contract treatment, so non-determinism is shared across conditions and the treatment effect is isolated—but the study uses a single model family, so it establishes direction and plausibility rather than a universal effect size.

Composed validity. Finally, the validity of a composed system is not the conjunction of the validities of its parts. Each chapter's result holds under its own assumptions, and those assumptions are not identical; an assumption satisfied in isolation may be violated in combination. The reference lifecycle of chapter 5 is accordingly claimed as a design coherent in its parts, with the interaction effects between them identified as precisely the questions a future end-to-end evaluation must measure. Naming that gap is part of the contribution: it tells a future evaluator where to look.

Chapter 7

Conclusion

7.1 The arc, restated

This thesis began from an observation about changing economics: as capable code-generating agents drive the marginal cost of a plausible implementation towards zero, the durable, scarce artefact of a software project ceases to be the code and becomes the precise, machine-checkable statement of what the code must do. Formal specifications have always promised to be that statement, and the machinery for *checking* programs against them—design by contract [45], the Java Modeling Language [38], and verifiers such as OpenJML [15] discharging obligations through Z3 [18]—has been mature for years. What was missing was an economical supply of the contracts themselves, and a place for them to do useful work across the lifecycle.

The programme answered that gap in three movements, and its shape is captured by its title. *Inference* makes specifications cheap: a heuristic static engine recovers sound, verifiable contracts from existing code at negligible authoring cost. *Composition* makes them reach: a weakest-precondition pass propagates those contracts across a call graph, and the same propagation, run across two versions of a dependency, turns behavioural version compatibility into a static, run-free analysis. *Integration* makes them load-bearing: the inferred, verified, compositional contract becomes the verification spine of an AI-native development lifecycle—at once the input an agent builds against and the oracle its output is checked against. Inference manufactures the artefact, composition extends its scope, and integration gives it somewhere to earn its keep; none of the three stands alone.

7.2 Contributions and research questions

The three contributions map onto the three research questions answered in chapter 6.

The first contribution (chapter 3) is the JML-Inferrer: a static engine that recovers JML preconditions, postconditions, loop invariants, and assignable clauses from unannotated Java by pattern-matching on the abstract syntax tree. Its output is validated end to end against OpenJML, so an emitted contract is not merely plausible but demonstrably discharges under extended static checking. A controlled comparison then shows that supplying these contracts to a language model measurably improves the unit tests it generates, relative to the same model working from code alone—because a

contract supplies the independent oracle a generated test needs [10]. This answers RQ1 affirmatively: even heuristic, partial, automatically inferred specifications are good enough to deliver downstream utility.

The second contribution (chapter 4) is a compositional weakest-precondition pass that strictly extends single-pass inference. Propagating a callee's contract into its callers recovers well-formed preconditions the local pass cannot—approximately 18,854 additional preconditions on the principal corpus, reported after well-formedness filtering, so vacuous or unsatisfiable-scope contracts are excluded from the count. Because the reasoning that recovers a caller's obligation from a callee's contract is exactly the reasoning that propagates a contract *change*, the identical mechanism, run differentially, computes the behavioural blast-radius of a dependency upgrade. This answers RQ2: a dependency change becomes a contract-diff event whose effect on clients can be checked rather than discovered in production—a behavioural notion of compatibility finer than the declared semantics of a version number [53].

The third contribution (chapter 5) is a verification-centric reference architecture in which one machine-checkable contract plays a dual role: input to the implementing agent and oracle for the verifier, with the verifier returning counterexamples that close the loop. It supplies the three things that current specification-driven agentic lifecycles—the AWS AI-Driven Development Lifecycle [3], GitHub's Spec Kit [29], and Kiro [4]—lack: a machine-checkable contract rather than prose intent, a closed counterexample-guided loop, and an oracle whose authority is independent of the code-writing agent. This answers RQ3 at the level of a design validated by instantiation of its components.

7.3 Limitations

Four limitations bound these claims, and the thesis is deliberate about each. The first is the boundary between a *verified* specification, which the checker has discharged, and a merely *well-formed* one, which is syntactically valid and scope-correct but has not passed that check; the compositional reach figure is a count of well-formed preconditions, not of verified ones, and the two are never conflated. The second is the partiality of heuristic inference: the engine recognises the structural patterns it was built for and stays silent elsewhere, so its fidelity is a rate over emitted contracts, not a coverage of all behaviour. The third is the reach of the verification substrate: Z3 timeouts, non-linear arithmetic, and quantified array theories place some true specifications beyond mechanical discharge, so silence from the checker is never a guarantee of safety. The fourth is that the reference pipeline of chapter 5 is designed and its components individually evidenced, but the integrated, agent-in-the-loop system has not been subjected to a controlled end-to-end trial. A fifth caveat cuts across the empirical results: the strongest quantitative findings rest on a single principal corpus, which establishes the effects in a realistic setting without establishing their generality.

7.4 Future work

Three directions follow directly. The first is industrial validation of the AI-native lifecycle: an action-research case study of the specification-centred workflow in a genuine development team [56, 63], framed to claim feasibility and realism rather than a controlled effect size, subject to Human Research Ethics Committee approval and to careful handling of confounds such as team experience and the simultaneous adoption of other tooling. The second is extending the compositional analysis—deeper inter-procedural reasoning and tighter modelling of standard-library and framework contracts—to raise the fraction of genuine incompatibilities the gate detects, moving it from sound-by-construction towards sound-and-comprehensive, and to replicate the findings on codebases beyond the principal corpus, including the messy legacy systems that most need retrofitted specifications [24]. The third is the direction deliberately deferred throughout: LLM-assisted inference at inference time, in which a language model proposes candidate contracts that the formal toolchain then certifies or rejects. The thesis has held this at arm's length precisely so that its fidelity claims rest on deterministic heuristics and mechanical verification; introducing a generative proposer while preserving that guarantee—the verifier remains the authority, the model only a source of candidates—is a natural and promising extension.

7.5 Closing

When an agent can regenerate an implementation cheaply, the implementation stops holding the value; what endures is the statement of what the software must do. This thesis has shown how to make that statement cheap, through inference that recovers verified contracts from existing code; how to make it reach, through compositional propagation that reasons about compatibility across a call graph; and how to make it load-bearing, as the verification spine of how software is built. The path from inference to integration is, in the end, a single argument: a formal specification, made cheap and placed at the centre, is the right foundation for software development in an age when the code can write itself but the intent still cannot.

Bibliography

[1] Wolfgang Ahrendt, Bernhard Beckert, Richard Bubel, Reiner Hähnle, Peter H. Schmitt, and Mattias Ulbrich. *Deductive Software Verification – The KeY Book: From Theory to Practice*, volume 10001 of *Lecture Notes in Computer Science*. Springer, 2016. doi: 10.1007/978-3-319-49812-6.

[2] Rajeev Alur, Rastislav Bodík, Garvit Juniwal, Milo M. K. Martin, Mukund Raghothaman, Sanjit A. Seshia, Rishabh Singh, Armando Solar-Lezama, Emina Torlak, and Abhishek Udupa. Syntax-guided synthesis. In *Formal Methods in Computer-Aided Design (FMCAD)*, pages 1–8, 2013. doi: 10.1109/FMCAD.2013.6679385.

[3] Amazon Web Services. AI-DLC: Ai-driven development lifecycle. https://github.com/awslabs/aidlc-workflows, 2025. Accessed: 2026-06-27.

[4] Amazon Web Services. Kiro: An AI IDE for spec-driven development. https://kiro.dev, 2025. Accessed: 2026-06-27.

[5] Glenn Ammons, Rastislav Bodík, and James R. Larus. Mining specifications. In *Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, pages 4–16, 2002. doi: 10.1145/503272.503275.

[6] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021. URL https://arxiv.org/abs/2108.07732.

[7] Ralph-Johan Back and Joakim von Wright. *Refinement Calculus: A Systematic Introduction*. Springer, New York, NY, 1998. doi: 10.1007/978-1-4612-1674-2.

[8] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. cvc5: A versatile and industrial-strength SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 13243 of *Lecture Notes in Computer Science*, pages 415–442. Springer, 2022. doi: 10.1007/978-3-030-99524-9_24.

[9] Mike Barnett, K. Rustan M. Leino, and Wolfram Schulte. The Spec# programming system: An overview. In *Construction and Analysis of Safe, Secure, and Interoperable Smart Devices*

(CASSIS 2004), volume 3362 of *Lecture Notes in Computer Science*, pages 49–69. Springer, 2005. doi: 10.1007/978-3-540-30569-9_3.

[10] Earl T. Barr, Mark Harman, Phil McMinn, Muzammil Shahbaz, and Shin Yoo. The oracle problem in software testing: A survey. *IEEE Transactions on Software Engineering*, 41(5): 507–525, 2015. doi: 10.1109/TSE.2014.2372785.

[11] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. APIDiff: Detecting API breaking changes. In *Proceedings of the 25th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 507–511, 2018. doi: 10.1109/SANER.2018.8330249.

[12] Aline Brito, Laerte Xavier, Andre Hora, and Marco Tulio Valente. Why and how Java developers break APIs. *Empirical Software Engineering*, 25(2):1115–1158, 2020. doi: 10.1007/s10664-019-09756-z.

[13] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. URL https://arxiv.org/abs/2107.03374.

[14] David R. Cok. OpenJML: JML for Java 7 by extending OpenJDK. In *NASA Formal Methods (NFM 2011)*, volume 6617 of *Lecture Notes in Computer Science*, pages 472–479. Springer, 2011. doi: 10.1007/978-3-642-20398-5_35.

[15] David R. Cok. OpenJML: Software verification for Java 7 using JML, OpenJDK, and Eclipse. *Electronic Proceedings in Theoretical Computer Science*, 149:79–92, 2014. doi: 10.4204/EPTCS.149.8.

[16] Matthias Cosler, Christopher Hahn, Daniel Mendoza, Frederik Schmitt, and Caroline Trippel. nl2spec: Interactively translating unstructured natural language to temporal logics with large language models. In *Computer Aided Verification (CAV 2023)*, volume 13965 of *Lecture Notes in Computer Science*, pages 383–396. Springer, 2023. doi: 10.1007/978-3-031-37703-7_18.

[17] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL)*, pages 238–252, 1977. doi: 10.1145/512950.512973.

[18] Leonardo de Moura and Nikolaj Bjørner. Z3: An efficient SMT solver. In *Tools and Algorithms for the Construction and Analysis of Systems (TACAS)*, volume 4963 of *Lecture Notes in Computer Science*, pages 337–340. Springer, 2008. doi: 10.1007/978-3-540-78800-3_24.

[19] Alexandre Decan, Tom Mens, and Philippe Grosjean. An empirical comparison of dependency network evolution in seven software packaging ecosystems. *Empirical Software Engineering*, 24(1):381–416, 2019. doi: 10.1007/s10664-017-9589-y.

[20] Edsger W. Dijkstra. Guarded commands, nondeterminacy and formal derivation of programs. *Communications of the ACM*, 18(8):453–457, 1975. doi: 10.1145/360933.360975.

[21] Edsger W. Dijkstra. *A Discipline of Programming*. Prentice-Hall, Englewood Cliffs, NJ, 1976.

[22] Madeline Endres, Sarah Fakhoury, Saikat Chakraborty, and Shuvendu K. Lahiri. Can large language models transform natural language intent into formal method postconditions? *Proceedings of the ACM on Software Engineering (FSE)*, 1(FSE):1889–1912, 2024. doi: 10.1145/3660791.

[23] Michael D. Ernst, Jeff H. Perkins, Philip J. Guo, Stephen McCamant, Carlos Pacheco, Matthew S. Tschantz, and Chen Xiao. The Daikon system for dynamic detection of likely invariants. *Science of Computer Programming*, 69(1–3):35–45, 2007. doi: 10.1016/j.scico.2007.01.015.

[24] Michael C. Feathers. *Working Effectively with Legacy Code*. Prentice Hall, 2004. ISBN 9780131177055.

[25] Cormac Flanagan and K. Rustan M. Leino. Houdini, an annotation assistant for ESC/Java. In *FME 2001: Formal Methods for Increasing Software Productivity*, volume 2021 of *Lecture Notes in Computer Science*, pages 500–517. Springer, 2001. doi: 10.1007/3-540-45251-6_29.

[26] Cormac Flanagan, K. Rustan M. Leino, Mark Lillibridge, Greg Nelson, James B. Saxe, and Raymie Stata. Extended static checking for Java. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, pages 234–245, 2002. doi: 10.1145/512529.512558.

[27] Gordon Fraser and Andrea Arcuri. EvoSuite: Automatic test suite generation for object-oriented software. In *Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE)*, pages 416–419, 2011. doi: 10.1145/2025113.2025179.

[28] Pranav Garg, Christof Löding, P. Madhusudan, and Daniel Neider. ICE: A robust framework for learning invariants. In *Computer Aided Verification (CAV)*, volume 8559 of *Lecture Notes in Computer Science*, pages 69–87. Springer, 2014. doi: 10.1007/978-3-319-08867-9_5.

[29] GitHub. Spec kit: Toolkit to help you get started with spec-driven development. https://github.com/github/spec-kit, 2025. Accessed: 2026-06-27.

[30] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969. doi: 10.1145/363235.363259.

[31] Frank Reyes García Islam, Gabriel Bartolomei, Martin Monperrus, and Benoit Baudry. Byam: Fixing breaking dependency updates with large language models. *arXiv preprint arXiv:2505.07522*, 2025. URL https://arxiv.org/abs/2505.07522.

[32] Dhanushka Jayasuriya, Sherlock A. Ranasinghe, Ewan Tempero, Kelly Blincoe, and Jens Dietrich. Understanding the impact of APIs behavioral breaking changes on client applications. *Proceedings of the ACM on Software Engineering*, 1(FSE):1238–1261, 2024. doi: 10.1145/3643782.

[33] Kamil Jezek and Jens Dietrich. API evolution and compatibility: A data corpus and tool evaluation. *Journal of Object Technology*, 16(4):2:1–2:23, 2017. doi: 10.5381/jot.2017.16.4.a2.

[34] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL https://openreview.net/forum?id=VTF8yNQM66.

[35] Adharsh Kamath, Aditya Senthilnathan, Saikat Chakraborty, Pantazis Deligiannis, Shuvendu K. Lahiri, Akash Lal, Aseem Rastogi, Subhajit Roy, and Rahul Sharma. Finding inductive loop invariants using large language models. arXiv preprint arXiv:2311.07948, 2023.

[36] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-C: A software analysis perspective. *Formal Aspects of Computing*, 27(3):573–609, 2015. doi: 10.1007/s00165-014-0326-7.

[37] Andrea Lattuada, Travis Hance, Chanhee Cho, Matthias Brun, Isitha Subasinghe, Yi Zhou, Jon Howell, Bryan Parno, and Chris Hawblitzel. Verus: Verifying Rust programs using linear ghost types. *Proceedings of the ACM on Programming Languages (OOPSLA)*, 7:286–315, 2023. doi: 10.1145/3586037.

[38] Gary T. Leavens, Albert L. Baker, and Clyde Ruby. Preliminary design of JML: A behavioral interface specification language for Java. *ACM SIGSOFT Software Engineering Notes*, 31(3): 1–38, 2006. doi: 10.1145/1127878.1127884.

[39] K. Rustan M. Leino. Dafny: An automatic program verifier for functional correctness. In *Logic for Programming, Artificial Intelligence, and Reasoning (LPAR-16)*, volume 6355 of *Lecture Notes in Computer Science*, pages 348–370. Springer, 2010. doi: 10.1007/978-3-642-17511-4_20.

[40] Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, et al. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, 2022. doi: 10.1126/science.abq1158.

[41] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023. URL https://arxiv.org/abs/2305.01210.

[42] Shuo Liu, Ying Zhang, and Lin Tan. Understanding LLM-driven test oracle generation. *arXiv preprint arXiv:2601.05542*, 2026. doi: 10.48550/arXiv.2601.05542.

[43] Lezhi Ma, Shangqing Liu, Yi Li, Xiaofei Xie, and Lei Bu. SpecGen: Automated generation of formal program specifications via large language models. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, 2025. doi: 10.1109/ICSE55347.2025.00129.

[44] Zohar Manna and Richard Waldinger. A deductive approach to program synthesis. *ACM Transactions on Programming Languages and Systems*, 2(1):90–121, 1980. doi: 10.1145/357084.357090.

[45] Bertrand Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992. doi: 10.1109/2.161279.

[46] Bertrand Meyer. *Object-Oriented Software Construction*. Prentice Hall, Upper Saddle River, NJ, 2 edition, 1997.

[47] Carroll Morgan. *Programming from Specifications*. Prentice Hall, Hemel Hempstead, UK, 2 edition, 1994.

[48] Shaikh Mostafa, Rodney Rodriguez, and Xiaoyin Wang. Experience paper: A study on behavioral backward incompatibilities of Java software libraries. In *Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 215–225, 2017. doi: 10.1145/3092703.3092721.

[49] Jeremy W. Nimmer and Michael D. Ernst. Static verification of dynamically detected program invariants: Integrating Daikon and ESC/Java. In *Proceedings of the First Workshop on Runtime Verification (RV)*, volume 55 of *Electronic Notes in Theoretical Computer Science*, pages 255–276, 2001. doi: 10.1016/S1571-0661(04)00256-7.

[50] Lina Ochoa, Thomas Degueule, Jean-Rémy Falleri, and Jurgen Vinju. Breaking bad? Semantic versioning and impact of breaking changes in Maven Central: An external and differentiated replication study. *Empirical Software Engineering*, 27(3):61, 2022. doi: 10.1007/s10664-021-10052-y.

[51] Ipek Ozkaya. Application of large language models to software engineering tasks: Opportunities, risks, and implications. *IEEE Software*, 40(3):4–8, 2023. doi: 10.1109/MS.2023.3248401.

[52] Carlos Pacheco, Shuvendu K. Lahiri, Michael D. Ernst, and Thomas Ball. Feedback-directed random test generation. In *Proceedings of the 29th International Conference on Software Engineering (ICSE)*, pages 75–84, 2007. doi: 10.1109/ICSE.2007.37.

[53] Tom Preston-Werner. Semantic versioning 2.0.0. https://semver.org/, 2013. Accessed 2026-06-27.

[54] Steven Raemaekers, Arie van Deursen, and Joost Visser. Semantic versioning and impact of breaking changes in the Maven repository. *Journal of Systems and Software*, 129:140–158, 2017. doi: 10.1016/j.jss.2016.04.008.

[55] Cedric Richter and Heike Wehrheim. Beyond postconditions: Can large language models infer formal contracts for automatic software verification? arXiv preprint arXiv:2510.12702, 2025.

[56] Per Runeson and Martin Höst. Guidelines for conducting and reporting case study research in software engineering. *Empirical Software Engineering*, 14(2):131–164, 2009. doi: 10.1007/s10664-008-9102-8.

[57] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. An empirical evaluation of using large language models for automated unit test generation. *IEEE Transactions on Software Engineering*, 50(1):85–105, 2024. doi: 10.1109/TSE.2023.3334955.

[58] Xujie Si, Hanjun Dai, Mukund Raghothaman, Mayur Naik, and Le Song. Learning loop invariants for program verification. In *Advances in Neural Information Processing Systems 31 (NeurIPS)*, pages 7762–7773, 2018. URL https://proceedings.neurips.cc/paper/2018/hash/65b1e92c585fd4c2159d5f33b5030ff2-Abstract.html.

[59] Armando Solar-Lezama. *Program Synthesis by Sketching*. PhD thesis, University of California, Berkeley, 2008. URL https://people.csail.mit.edu/asolar/papers/thesis.pdf.

[60] Armando Solar-Lezama, Liviu Tancau, Rastislav Bodík, Sanjit Seshia, and Vijay Saraswat. Combinatorial sketching for finite programs. In *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, pages 404–415, 2006. doi: 10.1145/1168857.1168907.

[61] Laerte Xavier, Aline Brito, Andre Hora, and Marco Tulio Valente. Historical and impact analysis of API breaking changes: A large-scale study. In *Proceedings of the 24th IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, pages 138–147, 2017. doi: 10.1109/SANER.2017.7884616.

[62] John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. SWE-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 37, 2024. URL https://arxiv.org/abs/2405.15793.

[63] Robert K. Yin. *Case Study Research and Applications: Design and Methods*. SAGE Publications, 6 edition, 2018.

[64] Ziyao Zhang, Chong Wang, Yanlin Wang, Ensheng Chen, and Zibin Zheng. LLM hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering (ISSTA)*, 2(ISSTA), 2025. doi: 10.1145/3728894.